



CHRONIQO

BACHELORPROJEKT

SW7BAC-01 BACHELORPROJEKT

Elev:

Mads Dittmann Villadsen

202209869 / AU715368

Vejleder:

Tommy Bjerre Nielsen

tbn@ece.au.dk

Antal anslag: 78.744 (32,8 sider)*

*Opgjort fra og med afsnit 1 (Indledning) til og med afsnit 10 (Fremtidigt arbejde), ekskl. figurtekster, tabeltekster, sidehoved og sidefod, jf. AU Engineerings projektvejledning.

MAJ 2026

Resumé

Kronisk sygdom medfører ikke blot fysiske udfordringer, men fører ofte til social isolation, idet svingende energiniveauer og uforudsigelige symptomer gør det vanskeligt at leve op til de sociale forventninger, som eksisterende platforme som Facebook og Instagram stiller. Dette projekt adresserer dette tomrum ved at designe og implementere Chroniqo - en responsiv webapplikation, der fungerer som et digitalt fællesskab for kronisk syge.

Systemets centrale innovation er muligheden for dagligt at registrere sin dagsform på en skala fra 1-5. Denne registrering er systemets omdrejningspunkt: når et lavt energiniveau detekteres, aktiveres et adaptivt brugerinterface (*Low Energy Mode*), der automatisk reducerer kognitiv belastning ved at skjule notifikationsbadges og erstatte uendeligt scroll med paginering.

Projektet er gennemført som enkeltmandsudvikler med udgangspunkt i ASE-modellen og en iterativ udviklingsproces. Den valgte teknologiske stak - Next.js (App Router), TypeScript, PostgreSQL via Prisma ORM, Auth.js og Upstash Redis - understøtter fire centrale designprincipper: Developer Experience, skalerbarhed, datasikkerhed og vedligeholdelse.

Systemet er valideret gennem en kombination af unit- og integrationstests implementeret med Jest, der bekræfter systemets kerneforretningslogik og forsvarsmekanismer. Resultatet er en fuldt fungerende MVP, der demonstrerer, hvordan softwareteknologi kan anvendes til aktivt at mindske social isolation hos kronisk syge.

Abstract

Chronic illness not only brings physical challenges but frequently leads to social isolation, as unpredictable energy levels and symptom fluctuations make it difficult to meet the social expectations imposed by mainstream platforms such as Facebook and Instagram. This project addresses this gap by designing and implementing Chroniqo - a responsive web application serving as a digital community for people living with chronic conditions.

The system's core innovation is a daily status indicator that allows users to register their energy level on a scale of 1-5. This registration drives the system's adaptive behavior: when a low energy level is detected, the interface activates *Low Energy Mode*, which automatically reduces cognitive load by hiding notification badges and replacing infinite scroll with pagination.

The project was carried out as a solo development effort, following the ASE model and an iterative development process. The chosen technology stack - Next.js (App Router), TypeScript, PostgreSQL via Prisma ORM, Auth.js, and Upstash Redis - supports four guiding design principles: Developer Experience, scalability, data security, and maintainability.

The system was validated through a combination of unit and integration tests implemented with Jest, confirming the core business logic and defensive mechanisms. The outcome is a fully functional MVP demonstrating how software engineering can actively reduce social isolation among people with chronic illness.

Forord

Denne rapport er udarbejdet som afslutningen på Bacheloruddannelsen i Softwareteknologi (Diplomingeniør) ved Aarhus School of Engineering (ASE), Aarhus Universitet, i foråret 2026.

Projektet er gennemført som enkeltmandsudvikler og udspringer af en personlig interesse i at anvende softwareteknologi til at løse en overset social udfordring: den isolation, der ofte følger med kronisk sygdom. Det har været et projekt, der på mange måder har stillet særlige krav til disciplin og prioritering, ikke mindst i lyset af egne helbredsmæssige udfordringer undervejs.

Jeg vil gerne rette en særlig tak til min vejleder, Tommy Bjerre Nielsen, for konstruktiv feedback og støtte gennem hele forløbet. En tak skal også lyde til Vokalo, hvor jeg har arbejdet sideløbende som student frontend-ingeniør, for forståelse og fleksibilitet på de dage, hvor presset har været for stort til at opretholde begge fronter.

Rapporten henvender sig til vejleder og eksaminatorer med interesse for moderne fullstack-webudvikling, systemdesign og brugercentreret softwareudvikling.

God læselyst.

Aarhus, maj 2026

Mads Dittmann Villadsen

Ordliste

Rapporten bevæger sig mellem et dansksprogligt domæne rettet mod slutbrugeren og en engelsksproget teknisk implementering. For at sikre en entydig kobling mellem forretningslogik og kildekode er der etableret et fastlagt begrebsapparat - et Ubiquitous Language (1) - som anvendes konsekvent gennem hele rapporten. Tabel 1 nedenfor angiver platformens centrale termer på begge sprog samt de overordnede tekniske begreber, der benyttes i det følgende.

Hver term introduceres første gang i rapporten med en fodnote, der i den digitale version fungerer som et klikbart bogmærke direkte tilbage til ordlisten. Fodnoterne er beregnet til hurtig navigation - som et integreret opslagsværk - så læseren ikke behøver at bladre manuelt for at genopfriske et begreb.

***Tabel 1:** Begrebsapparat og teknisk ordliste for Chroniqo-projektet. Tabellen oversætter rapportens og brugergrænsefladens danske termer til den underliggende engelske kodebase og definerer centrale tekniske koncepter, der anvendes gennem rapporten.*

Ordliste		
Begreb	Engelsk betegnelse	Forklaring
ACID	ACID (Atomicity, Consistency, Isolation, Durability)	Fire egenskaber der garanterer pålidelig behandling af databasetransaktioner: atomicitet sikrer at en transaktion enten gennemføres fuldt ud eller slet ikke, konsistens at databasen altid er i en gyldig tilstand, isolation at samtidige transaktioner ikke påvirker hinanden, og holdbarhed at gennemførte transaktioner overlever systemfejl.

Adskillelse af ansvar	Separation of Concerns (SoC)	Et designprincip, der fastslår, at hver del af et system kun skal håndtere ét veldefineret ansvarsområde.
Credential stuffing	Credential stuffing	En automatiseret angrebsform, hvor en angriber afprøver lister over kendte kombinationer af brugernavn og adgangskode - typisk fra tidligere databrud - mod en ny tjeneste. Rate limiting på autentificeringsendpoints er Chroniqos primære forsvar mod denne angrebstype.
Cron-job	Cron-job	En planlagt opgave, der automatisk eksekveres af operativsystemet eller en cloud-tjeneste ved faste tidsintervaller uden manuel indgriben. I Chroniqo anvendes to cron-jobs: ét til oprydning af udløbne samtaler (prune-chats) og ét til sletning af uverificerede brugerkonti (prune-unverified-users).
Cross-Site Request Forgery	CSRF	En angrebstype, hvor en ondsindet tredjepart narrer en autentificeret bruger til at sende uønskede anmodninger til et system, som brugeren allerede er logget ind på.
Cross-Site Scripting	XSS	En angrebstype, hvor ondsindet JavaScript injiceres i en webside og eksekveres i en anden brugers browser, typisk med henblik på at stjæle session-cookies eller brugerdata.

cuid (Collision-resistant Unique Identifier)	cuid	Et format for klientgenererede, kollisionsresistente unikke identifikatorer, der er designet til at være globalt unikke og uforudsigelige. I Chroniqo benyttes cuid-baserede primærnøgler på alle modeller - genereret server-side via Prismas <code>@default(cuid())</code> -direktiv - for at forhindre ID-enumerationsangreb, hvor en angriber sekventielt kan gætte andre brugers ressource-ID'er.
Dataoverførselsobjekt	DTO (Data Transfer Object)	Et objekt, der udelukkende benyttes til at transportere data mellem arkitekturlag. I Chroniqo håndhæver Zod-baserede DTO'er inputvalidering ved API-grænsefladen og forhindrer over-posting ved kun at acceptere eksplicit definerede felter fra klienten - felter som <code>role</code> modtages aldrig fra request-bodyen, men hentes udelukkende fra den server-side session.
Flerlagret arkitekturmønster	Layered Architecture	En arkitekturstruktur, der opdeler systemet i lag med adskilte ansvarsområder, f.eks. præsenterations-, logik- og data-lag.
Forældreløse data	Orphaned data	Datarækker i en database, der har mistet deres tilknytning til en overordnet entitet, typisk som følge af manglende sletningsregler.

Fælles fagsprog	Ubiquitous Language	Et fastlagt begrebsapparat, der sikrer, at kode og dokumentation anvender de samme termer konsekvent på tværs af et projekt.
Indlejrede svar	Nested replies	Kommentarer, der svarer på andre kommentarer. Begrænset til ét niveau i Chroniqo.
Kantnetværk	Edge Network	Et globalt distribueret netværk af servere placeret geografisk tæt på slutbrugeren. Muliggør eksekvering af kode med minimal latens inden anmodningen videresendes til den centrale applikationsserver. I Chroniqo benyttes Vercels Edge Network til at afvikle Next.js Middleware med rate limiting og i18n-routing.
Koblingstabel	Junction table	En databasetabel, der forbinder to andre tabeller via fremmednøgler for at modellere mange-til-mange-relationer.
Komorbiditet	Comorbidity	Tilstedeværelsen af én eller flere yderligere diagnoser eller lidelser hos en patient ved siden af den primære diagnose. Begrebet benyttes i rapporten for at motivere, at kronisk syge brugere hyppigt har ledsagende visuelle eller motoriske vanskeligheder, der yderligere skærper platformens behov for formel WCAG-certificering.

LCP (Largest Contentful Paint)	LCP (Largest Contentful Paint)	En Core Web Vitals-metrik, der måler den tid, der går fra en sides indlæsning påbegyndes, til det største synlige indholdselement er fuldt renderet i brugerens viewport. Benyttes i rapporten til at evaluere og sammenligne feedets indlæsningshastighed under simuleret og reel 4G-belastning.
Minimum Viable Product	Minimum Viable Product (MVP)	Den tidligste version af et produkt med tilstrækkelig funktionalitet til at validere kerneværdien over for brugerne.
NER-model	NER (Named Entity Recognition)	En teknik inden for naturlig sprogbehandling (NLP), der automatisk identificerer og klassificerer navngivne entiteter i fritext - f.eks. personnavne, telefonnumre og e-mailadresser. Nævnes i rapporten som et potentielt alternativ til Chroniqos nuværende regex-baserede PII-filter.
NoOps	NoOps (No Operations)	En driftsmodel, hvori al serverinfrastruktur er uddelegeret til managed cloud-tjenester, så udviklere ikke selv varetager serverpatch, skalering eller netværkskonfiguration. I Chroniqo realiseres NoOps via Vercel, Neon.tech og Upstash.

Optimistisk brugerflade	Optimistic UI	Et UX-mønster, hvor brugergrænsefladen øjeblikkeligt afspejler en brugerhandling, som om den er gennemført med succes - f.eks. ved at tilføje støtte til et opslag - og derefter tilbagerulles til den database-bekræftede tilstand, hvis serverkaldet returnerer en fejl.
Personhenførbare oplysninger	PII (Personally Identifiable Information)	Data, der direkte eller indirekte kan identificere en specifik fysisk person - f.eks. navn, e-mailadresse, telefonnummer eller CPR-nummer. I Chroniqo filtreres PII server-side via et regex-baseret filter, inden brugerdata videresendes til Gemini API.
Privacy by Design	Privacy by Design	Et GDPR-forankret princip, der foreskriver, at databeskyttelse skal integreres i systemets arkitektur fra begyndelsen - frem for at blive tilføjet efterfølgende. I Chroniqo realiseres princippet bl.a. via on-Delete: Cascade på alle brugerrelaterede modeller og via server-side PII-filtrering i AI-integrationen.

Proof Key for Code Exchange	PKCE	En sikkerhedsudvidelse til OAuth 2.0 Authorization Code-flowet, der beskytter mod angreb, hvor authorization codes opsnappes under omdirigering. Klienten genererer et tilfældigt code verifier og sender kun en hashværdi heraf (code challenge) til autoriseringsserveren; selve code verifier-hemmeligheden vedlægges først ved den efterfølgende token-udveksling.
Reaktivt datafetching-bibliotek	SWR (Stale-While-Revalidate)	Et React-bibliotek fra Vercel til datahentning på klientsiden. Viser øjeblikkeligt cachelagrede data (stale), mens friske data hentes i baggrunden (revalidate), hvilket giver en hurtig og responsiv brugeroplevelse. I Chroniqo benyttes SWR til short-polling i chat-funktionaliteten samt til feed-datahentning, og <code>globalMutate</code> anvendes til øjeblikkelig cache-invalidering på tværs af samtalevisninger ved registrering af Dagsform.
Sammensat unikhedsbetingelse	Composite Unique Constraint	En databasebetingelse, der sikrer, at kombinationen af værdier i to eller flere kolonner er unik på tværs af alle rækker i en tabel. I Chroniqo anvendes <code>@@unique([userId, date])</code> på <code>DailyStatus</code> -modellen til at håndhæve, at én bruger maks. kan registrere én dagsform pr. kalenderdag - uanset klokkeslæt.

Server-side rendering	SSR (Server-Side Rendering)	En renderingsteknik, hvori HTML genereres på serveren ved hvert request og sendes færdigrenderet til klienten - frem for at lade JavaScript bygge DOM'en i browseren. Reducerer Time to First Byte og forbedrer LCP, da det første indhold er tilgængeligt i den initiale HTML-respons. I Chroniqo identificeres SSR-migrering af feed-komponenten som en konkret optimeringsopgave til næste iteration.
Short-polling	Short-polling	En kommunikationsteknik, hvor klienten sender gentagne HTTP-forespørgsler til serveren ved faste intervaller for at tjekke, om der er nye data. I Chroniqo benyttes short-polling via SWR som et bevidst MVP-valg frem for stateful WebSockets, idet den serverless infrastruktur ikke understøtter persistente TCP-forbindelser.
Think-Aloud-test	Think-Aloud test	En kvalitativ brugertest-metode, hvor deltagerne verbalt beskriver deres tanker og handlinger, mens de løser en opgave. Metoden giver direkte indsigt i kognitive udfordringer og usability-problemer, der ikke lader sig afdække via spørgeskema alene. Benyttes i Chroniqos brugertest-protokol til at observere reaktionen på <i>Low Energy Mode</i> .

Indholdsfortegnelse

Resumé.....	i
Abstract	ii
Forord.....	iii
Ordliste.....	iv
Figurliste	xvi
Tabelliste	xvii
1. Indledning	1
1.1. Motivation og baggrund.....	2
1.1.1. Problemet: social isolation ved kronisk sygdom.....	2
1.1.2. Manglen på eksisterende løsninger	2
1.1.3. Målet: Chroniqo som digitalt frirum.....	3
1.2. Problemformulering	4
1.2.1. Problemanalyse	4
1.2.2. Hovedspørgsmål.....	4
1.2.3. Underspørgsmål	4
1.3. Afgrænsning	5
1.3.1. Platform.....	5
1.3.2. Funktionalitet	5
1.3.3. Målgruppe	5
1.3.4. Medicinsk rådgivning og ansvar	6
1.4. Læsevejledning	6
2. Kravspecifikation	8
2.1. Aktøranalyse.....	9
2.2. Epics og User Stories	11
2.2.1. User Story (ID: US2.1): Dagsformsregistrering	11
2.2.2. User Story (ID: US2.13): Adaptivt brugerinterface (Low Energy Mode)	11
2.2.3. User Story (ID: US3.10): Anonym interaktion	11
2.3. MoSCoW-prioritering	12
2.3.1. Metodevalg.....	12

2.3.2. Kategorisering af funktioner	12
2.3.3. Begrundelse for prioriteringen	13
2.4. Funktionelle og ikke-funktionelle krav (FURPS+).....	14
2.4.1. Kvalitetskrav (URPS)	14
2.4.2. Designrestriktioner og øvrige krav (+).....	15
2.5. Accepttestspecifikation (Gherkin-scenarier).....	16
2.5.1. Scenarie 1: Registrering af Dagsform (Epic 2 - Must)	16
2.5.2. Scenarie 2: Anonym interaktion i fællesskaber (Epic 3 + 4 - Must).....	16
2.6. Delkonklusion for kravspecifikation.....	17
3. Metode og proces	18
3.1. Valg af udviklingsmetode.....	19
3.2. Projektstyring og værktøjer.....	20
3.2.1. Kanban-metoden	20
3.2.2. Trello til opgavestyring	20
3.2.3. Git til versionsstyring og GitLab som remote repository	21
3.3. Tidsplan.....	22
3.4. Brug af generativ AI.....	23
4. Analyse.....	24
4.1. Domæneanalyse	25
4.1.1. Kernekoncepter og forretningsregler	25
4.1.2. Fælles fagsprog (Ubiquitous Language).....	26
4.2. Risikoanalyse	27
4.2.1. R1: Personlige helbredsudfordringer (ryg og/eller fatigue).....	28
4.2.2. R13: Sikkerhedshuller (OAuth 2.0-implementation).....	28
4.2.3. R14: Problemer relateret til systemets deployment	28
4.2.4. R9: Afhængighed af eksterne tjenester	28
4.3. Teknologianalyse.....	29
4.3.1. Teknologivalg.....	29
4.3.2. Arkitektoniske overvejelser og risikohåndtering	31
4.4. Delkonklusion for analyse	32
5. Systemdesign og arkitektur.....	33

5.1. Systemarkitektur (C4-modellen).....	34
5.2. Databasedesign.....	37
5.3. Applikationsmodel (softwaredesign)	40
5.4. Dynamisk adfærd (sekvensdiagrammer)	41
5.4.1. Registrering af Dagsform (US2.1)	41
5.5. API-design (grænseflader)	43
5.6. Delkonklusion for systemdesign og arkitektur	44
6. Design og implementering	45
6.1. UI/UX-design og visuel identitet	46
6.1.1. Designsystem og tilgængelighed (Accessibility)	46
6.1.2. Logo og visuelt udtryk	48
6.1.3. Dagsformsindikatoren i praksis	50
6.1.4. Det adaptive UI (Low Energy Mode)	53
6.2. Databasedesign og implementering	56
6.2.1. Fysisk implementering i Prisma.....	57
6.2.2. Migrering og databasehosting	58
6.3. Softwaredesign og kodestruktur.....	59
6.3.1. Mappestruktur og komponentarkitektur.....	59
6.3.2. Implementering af autentificering.....	60
6.3.3. Håndtering af rate limiting og JWT-revokation	63
6.4. Automatiseret deployment	66
6.5. Delkonklusion for design og implementering.....	67
7. Test og validering	68
7.1. Teststrategi (V-modellen).....	69
7.2. Automatiserede tests (Unit og Integration)	71
7.3. Databasetest og dataintegritet	73
7.4. Accepttest - manuel systemtest	74
7.4.1. Dagsformsregistrering (US2.1 - Must)	75
7.4.2. Anonym interaktion (US3.10 - Must)	75
7.5. Test af ikke-funktionelle krav (FURPS+)	76
7.5.1. Usability (U)	78

7.5.2. Reliability (R)	78
7.5.3. Performance (P)	78
7.5.4. Supportability (S).....	78
7.6. Brugertest (validering)	79
7.7. Delkonklusion for test og validering.....	80
8. Diskussion.....	81
8.1. Kravopfyldelse og MVP-realiserings	82
8.1.1. FURPS+-krav: konstruktiv evaluering af de delvist opfyldte krav.....	84
8.1.2. Den ikke-eksekverede brugertest	84
8.2. Tekniske kompromiser og arkitektoniske refleksioner	85
8.2.1. Architectural Simplification: beslutningen der holdt	85
8.2.2. Short-polling til chat: et bevidst kompromis der holdt sit løfte	85
8.2.3. Kommentar-nesting begrænset til ét indlejningsniveau	86
8.2.4. Minispil: en uventet designmæssig udfordring	86
8.2.5. PII-filteret i Niqo: pragmatisk, men ikke ufejlbarligt	87
8.3. Proces og projektstyring.....	88
8.3.1. Tidsplan og fremdrift	88
8.3.2. Trello-board som dokumentation for færdiggørelse	89
8.3.3. GitLab-aktivitetsgraf som dokumentation for udviklingsrytmen	90
8.3.4. ASE-modellen i praksis.....	91
8.4. Helbredsmæssige udfordringer og projektets gennemførelse.....	92
9. Konklusion	94
10. Fremtidigt arbejde	96
10.1. Eksekvering af brugertest med målgruppen.....	97
10.2. Opgradering til Managed Realtime-chatinfrastruktur.....	98
10.3. Native mobilapplikation.....	99
10.4. WCAG 2.1 AA-certificering	100
10.5. Niqo med dagsforms-integration og opt-in langtidshukommelse.....	101
10.6. Formel load-test og optimering af server-side rendering i feed	102
11. Referencer	103
12. Bilag	108

Figurliste

Figur 1: ASE-modellens overordnede udviklingsramme.....	7
Figur 2: Aktørkontekstdiagram for Chroniqo	9
Figur 3: Trello-board med Kanban-flow	21
Figur 4: Git-branching-strategi	22
Figur 5: Gantt-diagram over projektets tidsplan og faser	23
Figur 6: Konceptuel domænemodel (UML-klassediagram)	26
Figur 7: Risikovurderingsmatrix med 15 identificerede risici	28
Figur 8: C4 Container Diagram (Level 2).....	36
Figur 9: ER-diagram for kommunikations- og kernerdomænet.....	39
Figur 10: Design-klassediagram for systemets kernerdomæne.....	41
Figur 11: Sekvensdiagram - Registrering af Dagsform (US2.1).....	43
Figur 12: Platformens dual-theme-farvepalet	47
Figur 13: Interaktiv Figma-prototype af landingssiden	48
Figur 14: Chroniqos monogramlogo i Light- og Dark Mode	49
Figur 15: Dagsformens fem farvekodede energiniveauer	50
Figur 16: Dagsformsinterceptor ved sessionstart.....	52
Figur 17: Profilside med aktiv dagsformsring	53
Figur 18: Feedvisning i standardtilstand	55
Figur 19: Feedvisning i Low Energy Mode	56
Figur 20: Prisma-skema - tre centrale implementeringsbeslutninger	57
Figur 21: Sekvensdiagram - OAuth 2.0 PKCE-flow med Google.....	62
Figur 22: Sekvensdiagram - Rate limiting-flow	65
Figur 23: V-modellen med Chroniqos fire testniveauer.....	70
Figur 24: GitLab CI/CD-pipeline - succesfuld kørsel.....	72
Figur 25: Trello-board ved afleveringstidspunktet.....	91
Figur 26: GitLab-aktivitetsgraf for projektperioden	92

Tabelliste

Tabel 1: Begrebsapparat og teknisk ordliste for Chroniqo-projektet	iv
Tabel 2: Beskrivelse af identificerede aktører opdelt i primære og sekundære/supporting	10
Tabel 3: MoSCoW-fordeling af 68 User Stories på tværs af fire Epics	12
Tabel 4: Oversigt over designrestriktioner og rammevilkår (FURPS+)	15
Tabel 5: Chroniqos teknologistak fordelt på kategori og komponent	31
Tabel 6: Samlet kodedækningsgrad opnået ved kørsel af testsuiten	73
Tabel 7: Acceptteststatus for alle 68 User Stories fordelt på fire Epics	75
Tabel 8: Teststatus for samtlige ikke-funktionelle krav (FURPS+)	77
Tabel 9: Repræsentativt udsnit af verificerede User Stories på tværs af Epics	84

1. Indledning

De fleste sociale platforme er designet til brugere med konstant energi og konstant tilgængelighed. For de mange mennesker, hvis kroniske sygdom gør netop dette til en umulighed, er de eksisterende platformes designvalg ikke neutrale - de er ekskluderende. Det er dét tomrum, Chroniqo er opstået til at udfylde, og det er dét tomrum, rapporten systematisk undersøger, designer og evaluerer mod.

1.1. Motivation og baggrund

Udviklingen af Chroniqo er drevet af et ønske om at anvende softwareteknologi til at løse en overset social udfordring: den isolation, der ofte følger med kroniske lidelser.

1.1.1. Problemet: social isolation ved kronisk sygdom

Kroniske lidelser påvirker millioner af mennesker globalt og medfører ofte udfordringer, der rækker langt ud over de rent fysiske symptomer. Studier viser, at personer med kroniske smerter eller langvarig sygdom har en markant højere risiko for social isolation og ensomhed (2). Denne isolation skyldes ofte usynlige faktorer som svingende energiniveauer (fatigue), uforudsigelige smerteforværringer og en følelse af ikke at kunne leve op til de sociale forventninger, som det omkringliggende samfund stiller (3).

Kronisk sygdom er ikke kun en fysisk belastning, men en kompleks tilstand, der påvirker patientens samlede livskvalitet. Ifølge Sundhedsstyrelsen lever hver femte dansker med kroniske smerter, hvilket ofte fører til nedsat arbejdsevne og social tilbagetrækning (4). Forskning viser, at social isolation fungerer som en forstærkende faktor for både de fysiske og emotionelle symptomer; jo mere isoleret en patient føler sig, desto sværere bliver det at håndtere de faktiske smerter (2). Denne form for isolation er ofte usynlig, idet patienten trækker sig socialt, fordi de sociale forventninger ikke harmonerer med deres svingende energiniveau og sygdomsfluktuationer.

1.1.2. Manglen på eksisterende løsninger

Eksisterende sociale platforme som Facebook eller Instagram er designet ud fra præmissen om konstant aktivitet og "det perfekte liv" (5). For en person med en kronisk lidelse kan disse platforme virke ekskluderende, da de ikke tager højde for brugerens dagsform.

Der mangler et digitalt værktøj, hvor det er socialt acceptabelt at have "dårlige dage", og hvor interaktionen er designet til at kræve minimalt mentalt og socialt overskud. Systematiske reviews viser, at støtte fra andre i samme situation har en afgørende positiv effekt på evnen til at mestre sin sygdom, men at generelle sociale medier ofte fejler i at facilitere denne specifikke form for støtte uden at skabe socialt pres (6). Der er med andre ord et tomrum mellem de behov, kronisk syge har for fællesskab, og de krav til energi, som nuværende sociale medier stiller.

1.1.3. Målet: Chroniqo som digitalt frirum

Målet med dette projekt er at skabe Chroniqo - en platform, der fungerer som et trygt og fleksibelt fællesskab. Ved at lade brugerens dagsform være det centrale element i brugerprofilen, kan brugere signalere deres aktuelle overskud uden at skulle forklare sig. Chroniqo skal ikke blot være endnu et socialt medie, men et ingeniørmæssigt gennemtænkt værktøj, der fjerner barriererne for social kontakt og skaber rammerne for et fællesskab baseret på gensidig forståelse og empati (3).

1.2. Problemformulering

Det skitserede problemfelt rejser spørgsmålet om, hvorvidt softwareteknologi konkret kan bidrage til at mindske den sociale isolation, mange kronisk syge oplever. For at give undersøgelsen en klar retning præciseres udfordringen i det følgende som et hovedspørgsmål med tilhørende underspørgsmål, der tilsammen udgør det undersøgelsesmæssige fundament for projektet.

1.2.1. Problemanalyse

Eksisterende software til social kontakt overser ofte det tomrum, der opstår, når en brugers fysiske tilstand forhindrer aktiv deltagelse. Ved at analysere målgruppens behov for asynkron kommunikation kan man identificere tekniske features, der kan mindske isolation. Den centrale udfordring er at transformere disse bløde brugerbehov til konkrete, målbare softwarekrav i en brugertilpasset responsiv løsning.

1.2.2. Hovedspørgsmål

Hvordan kan softwareteknologi anvendes til at mindske social isolation hos kronisk syge ved at udvikle en webapplikation, der adapterer dele af brugeroplevelsen efter individets aktuelle dagsform?

1.2.3. Underspørgsmål

- Hvordan kan dagsforms-indikatorer implementeres som en central logik i systemet, der automatisk adapterer brugergrænsefladen (f.eks. ved at reducere visuel støj og notifikationer), uden at øge den kognitive kompleksitet for slutbrugeren?
- Hvilke tekniske valg i frontend-udviklingen (f.eks. komponentbaserede frameworks som React) understøtter bedst en responsiv brugerflade til denne målgruppe?
- Hvordan kan man validere, at den udviklede løsning reelt imødekommer de opstillede krav til et lavenergi-fællesskab gennem både tekniske og brugerorienterede evalueringsmetoder?

1.3. Afgrænsning

Problemformuleringen afspejler et bredt og komplekst problemrum, der i princippet kunne adresseres fra mange retninger. For at sikre metodisk konsistens er der truffet en række bevidste afgrænsninger, der fokuserer projektets samlede arbejde på de områder, der vurderes som mest centrale for systemets værdiskabelse, inden for projektets tidsmæssige rammer.

1.3.1. Platform

Projektet afgrænses til udviklingen af en responsiv webapplikation. Udvikling af native mobilapplikationer til iOS og Android er bevidst fravalgt for at sikre et realistisk omfang og fokusere på dybdegående softwareudvikling inden for projektets tidsramme. Webapplikationen designes med adaptiv layout for at sikre tilfredsstillende brugervenlighed på både desktop- og mobile enheder.

1.3.2. Funktionalitet

Projektet fokuserer på kernefunktionalitet for et asynkront socialt fællesskab. Dette inkluderer brugerprofiler, relationer mellem brugere (f.eks. venskabsrelationer), deling af opslag, tekstbaseret chat, understøttelse af simple asynkrone minispil i beskeder samt en sessionbaseret AI-støttebot, Niqo, for at facilitere social interaktion og emotionel støtte med lav kognitiv indsats. Funktioner med højt energikrav, såsom realtidsvideo eller stemmeopkald, er derimod bevidst fravalgt for at beskytte målgruppen. Systemet implementeres som en MVP¹ med fokus på understøttelse af interaktion med lavt energikrav frem for at matche funktionsniveauet i eksisterende sociale platforme.

1.3.3. Målgruppe

Selvom platformen principielt kan have relevans for flere brugergrupper, afgrænses den indledende analyse og evaluering til personer med kroniske smertetilstande eller fatiguerelaterede lidelser for at sikre et konsistent og realistisk evalueringsgrundlag.

¹ Minimum Viable Product (MVP).

1.3.4. Medicinsk rådgivning og ansvar

Platformen er designet som et socialt støtteværktøj og ikke som et system til medicinsk rådgivning, diagnosticering eller behandling. I det omfang at projektet inkluderer AI-baserede funktioner, herunder en chatbaseret støttebot, er disse udelukkende tiltænkt en støttende og empatisk rolle og må ikke betragtes som sundhedsfaglig vejledning. AI interaktion på platformen, herunder brugerindhold, skal forstås som erfaringsbaseret og ikke-professionel.

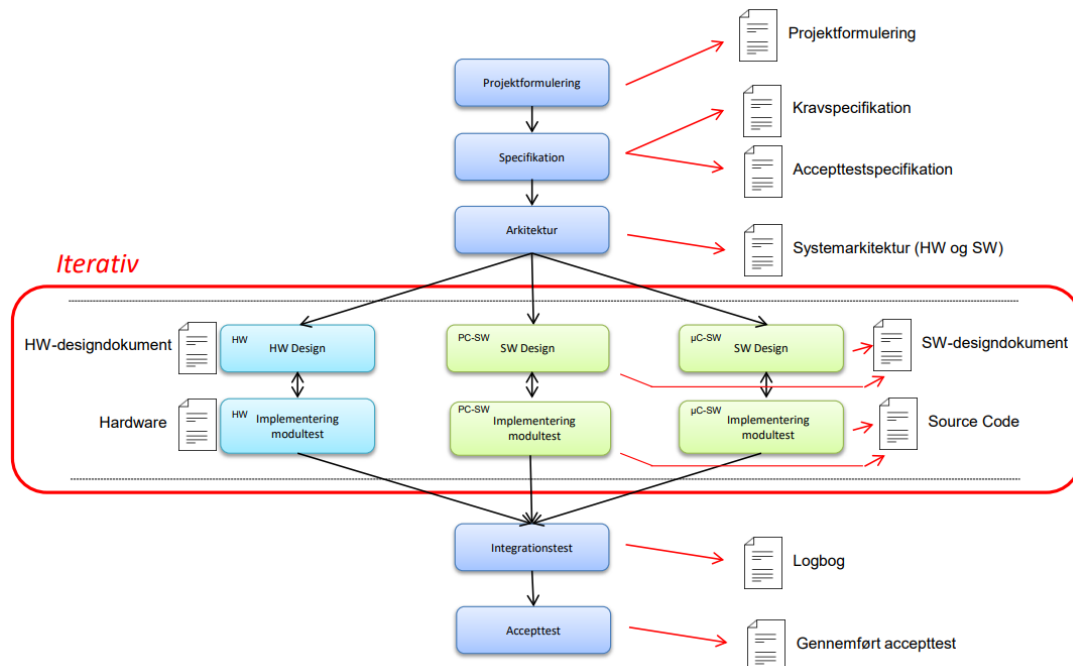
1.4. Læsevejledning

Rapporten er struktureret efter ASE-modellen (Semesterprojektmodellen) (7), som er en iterativ og use case-drevet softwareudviklingsproces. Denne struktur afspejles i rapportens opbygning, som overordnet kan beskrives som følger:

- **Afsnit 1 - Indledning:** Motivation, problemformulering, afgrænsning, medicinsk ansvar, målgruppe og funktionalitet.
- **Afsnit 2 - Kravspecifikation:** Aktører, User Stories, MoSCoW-prioritering, funktionelle og ikke-funktionelle krav, accepttest.
- **Afsnit 3 - Metode og proces:** Valg af udviklingsmetode, projektstyring, tidsplan, anvendte værktøjer samt brug af generativ AI.
- **Afsnit 4 - Analyse:** Risikoanalyse, teknologianalyse og domæneanalyse.
- **Afsnit 5 - Systemdesign og arkitektur:** Systemarkitektur og domænemodel.
- **Afsnit 6 - Design og implementering:** UI/UX-design, databasedesign og softwaredesign.
- **Afsnit 7 - Test:** Teststrategi og testresultater.
- **Afsnit 8 - Diskussion:** Evaluering i forhold til krav, refleksion over proces og helbredsmæssige udfordringer.
- **Afsnit 9 - Konklusion:** Besvarelse af projektets problemformulering og opsummering af de overordnede resultater.
- **Afsnit 10 - Fremtidigt arbejde:** Perspektivering og forslag til videreudvikling af projektet.
- **Afsnit 11 - Referenceliste:** Kildeliste refereret numerisk efter Vancouver-standardens med kildehenvisninger i formatet (1).
- **Afsnit 12 - Bilag:** Oversigt over projektets tilhørende referencedokumenter (Bilag 1-9).

Kildehenvisninger følger Vancouver-standarden og nummereres i den rækkefølge, de første gang optræder i rapporten. Da bilag indgår i samme EndNote-bibliotek som øvrige kilder, kan der forekomme spring i nummereringen i referencelisten - eksempelvis fra (7) til (9) - fordi de mellemliggende numre er tildelt bilag, der er oplistet særskilt under afsnit 12.

For at give et visuelt overblik over den iterative proces og de tilhørende leverancer, er ASE-modellen illustreret i Figur 1:



Figur 1: Visuel repræsentation af ASE-modellen (7), som udgør den overordnede udviklingsramme for projektet. Modellen illustrerer et flow fra de indledende krav- og arkitekturfaser, der leder ind i en central iterativ kerne (markeret med en rød boks). Her gentages design, implementering og modultest i cyklusser. Da Chroniqo udelukkende er en web-applikation, følges primært modellens midterste softwarespor (PC-SW), hvorefter iterationerne samles og afsluttes i de endelige integrations- og accepttests.

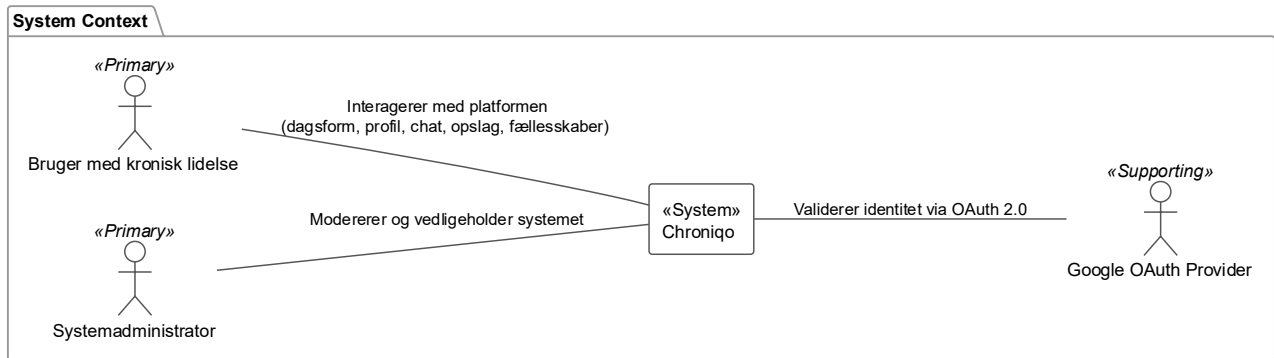
2. Kravspecifikation

Kravspecifikationen er den tekniske oversættelse af det menneskelige behov identificeret i afsnit 1. Aktøranalyse, User Stories, MoSCoW-prioritering, FURPS+-kvalitetskrav og Gherkin-baserede accepttestscenarier transformerer brugerbehovene til verificerbare systemkrav med direkte sporbarhed til teststrategien i afsnit 7.

Den fulde kravspecifikation - herunder samtlige User Stories, FURPS+-krav og Gherkin-scenarier - fremgår af Bilag 1: Kravspecifikation (8).

2.1. Aktøranalyse

Chroniqos kontekst fastlægges ved at identificere de eksterne aktører, som systemet udveksler information med. Figur 2 viser systemet afgrænset som en black box i relation til dets omgivelser:



Figur 2: Aktørkontekstdiagram for Chroniqo, der illustrerer systemets afgrænsning som en afgrænset black box samt afhængigheder mellem systemet og de tre identificerede aktører: den primære bruger med kronisk lidelse, systemadministratoren og den understøttende Google OAuth Provider. Stregerne angiver forbindelsen mellem aktørerne og systemet; de primære aktører anvender systemet til henholdsvis platformsinteraktion og systemadministration, mens Chroniqo anvender Google OAuth Provider til identitetsvalidering.

Tabel 2 beskriver de tre identificerede aktører. Systemet skelner mellem primære aktører, der tager initiativ til interaktion for at opnå et mål, og sekundære/supporting aktører, der leverer en service til systemet:

Tabel 2: Beskrivelse af de identificerede aktører, opdelt i primære og sekundær/supporting, med fokus på deres ansvarsområder og interaktion med Chroniqo.

Aktørkontekstbeskrivelse		
Aktør	Type	Beskrivelse
Bruger med kronisk lidelse	Primær	Systemets hovedaktør. Interagerer med platformen for at administrere sin profil, angive dagsform, oprette opslag, chatte og deltage i eller administrere fællesskaber med henblik på social støtte.
Systemadministrator	Primær	Ansvarlig for platformens drift og moderation. Aktøren har udvidede rettigheder til at overvåge brug, bandlyse brugere samt moderere fællesskaber og opslag for at sikre et trygt miljø.
Google OAuth Provider	Sekundær / Supporting	Et eksternt autentificeringssystem, der verificerer brugerens identitet via OAuth 2.0-protokollen og returnerer valideret brugerinformation til Chroniqo.

2.2. Epics og User Stories

Systemets funktionelle krav er struktureret i Epics og tilhørende User Stories. Epics repræsenterer de overordnede funktionelle hovedområder i systemet, mens User Stories specificerer konkrete krav fra brugerens perspektiv. Denne struktur understøtter en brugercentreret og iterativ udviklingsproces, hvor funktionaliteten kan implementeres og prioriteres trinvis. Samtidig bidrager opdelingen i Epics til at skabe et klart overblik over systemets arkitektur og centrale ansvarsområder.

Systemet er inddelt i følgende fire Epics: **Autentificering & Profil (Epic 1)**, der håndterer sikker adgang og brugeroplysninger; **Dagsform & Adaptivt UI (Epic 2)**, der udgør systemets centrale funktionalitet; **Socialt Fællesskab (Epic 3)**, der understøtter asynkron social interaktion; og **Platform Moderation (Epic 4)**, der sikrer et trygt og kontrolleret digitalt fællesskab. Den fulde specifikation af samtlige 68 User Stories fremgår af Bilag 1 (8).

Nedenfor præsenteres tre User Stories udvalgt for at illustrere systemets centrale og Chroniqo-specifikke funktionelle adfærd. Disse er valgt, fordi de repræsenterer tre distinkte Epic-områder og spores videre i rapportens efterfølgende afsnit om design, implementering og test:

2.2.1. User Story (ID: US2.1): Dagsformsregistrering

Som en bruger vil jeg kunne registrere min dagsform ved første login hver 24. time, **så jeg** kan få en platformoplevelse, der passer til mit aktuelle funktionsniveau.

2.2.2. User Story (ID: US2.13): Adaptivt brugerinterface (Low Energy Mode)

Som en bruger vil jeg kunne opleve, at systemet automatisk skjuler potentielt stressende elementer (som ulæste notifikationstællere og uendeligt scroll) på dage med lav dagsform (lavt energiniveau), **så jeg** kan deltage i fællesskabet uden at blive kognitivt overvældet.

2.2.3. User Story (ID: US3.10): Anonym interaktion

Som en bruger vil jeg kunne vælge at fremstå anonym ved oprettelse af opslag i fællesskaber, samtidig med at moderatorer og administratorer fortsat kan identificere mig, **så jeg** kan dele følsomme erfaringer uden at eksponere min identitet.

2.3. MoSCoW-prioritering

MoSCoW-metoden (9) anvendes til at strukturere og prioritere systemets krav med henblik på at understøtte udviklingen af Chroniqos MVP. Prioriteringen skaber et klart overblik over funktionernes betydning, implementeringsrækkefølge samt deres relative værdi i systemets samlede kravgrundlag.

2.3.1. Metodevalg

MoSCoW-prioriteringen anvender i dette projekt kategorierne *Must*, *Should* og *Could*. Kategorien *Won't* er ikke eksplicit anvendt, da strategiske fravalg i stedet håndteres gennem projektets afgrænsning i afsnit 1.3, hvor funktioner uden for MVP'ens scope beskrives.

2.3.2. Kategorisering af funktioner

Must-haves dækker funktioner, der sikrer grundlæggende datasikkerhed, systemadfærd og social interaktion - krav hvis fravær ville gøre systemet ubrugeligt eller udgøre en reel risiko for målgruppens sikkerhed og privatliv.

Should-haves understøtter og styrker brugeroplevelsen, mens *Could-haves* kan implementeres i senere iterationer uden at påvirke systemets fundamentale funktionalitet.

Tabel 3 viser den samlede MoSCoW-fordeling på tværs af alle fire Epics:

Tabel 3: MoSCoW-fordeling af samtlige 68 User Stories fordelt på de fire Epics, med angivelse af antallet af krav i kategorierne *Must*, *Should* og *Could*.

MoSCoW-fordeling af samtlige 68 User Stories fordelt på de fire Epics				
Epic	Must	Should	Could	Total
Epic 1: Autentificering & Profil	10	7	2	19
Epic 2: Dagsform & Adaptivt UI	8	6	2	16
Epic 3: Socialt Fællesskab	7	7	5	19
Epic 4: Platform Moderation	9	3	2	14
I alt	34	23	11	68

2.3.3. Begrundelse for prioriteringen

Fordelingen på 34 Must-have-krav ud af 68 i alt afspejler, at systemet er et socialt medie med en ikke-triviel mængde kerneforretningslogik: autentificering, privatlivsregler, dagsformsmekanik og moderationsværktøjer er alle uundværlige for et ansvarligt og trygt system. Should- og Could-kravene sikrer, at MVP'en kan leveres inden for projektets tidsramme, samtidig med at platformen bevarer et klart udviklingsgrundlag for kommende iterationer.

2.4. Funktionelle og ikke-funktionelle krav (FURPS+)

For at sikre systemets kvalitet og robusthed er kravene kategoriseret efter FURPS+-modellen, der adskiller de funktionelle krav fra de overordnede kvalitetsattributter (10). De funktionelle krav, som beskriver, hvad systemet skal gøre, er behandlet gennem Epics og User Stories i afsnit 2.2 og Bilag 1 (8).

2.4.1. Kvalitetskrav (URPS)

Nedenfor fremhæves ét nøglekrav per kvalitetskategori for at illustrere de mest afgørende kvalitetsbeslutninger i systemet. Den fulde liste med alle 13 ikke-funktionelle krav fremgår af Bilag 1 (8).

2.4.1.1. Usability

UR1 (M): Navigation til kernefunktioner (f.eks. dagsform eller chat) skal kunne udføres med maksimalt 2-3 klik fra forsiden for at mindske kognitiv belastning hos målgruppen.

2.4.1.2. Reliability

RR1 (M): Brugerens dagsformsdata og noter skal persisteres med 100 % integritet, så historikken altid er korrekt - dette er en ufravigelig forudsætning for systemets troværdighed over for målgruppen.

2.4.1.3. Performance

PR1 (S): Indlæsning af det personlige feed skal ske inden for 2 sekunder ved en standard 4G-forbindelse.

2.4.1.4. Supportability

SR1 (M): Kildekoden skal være modulært opbygget (komponentbaseret f.eks. i React), så nye funktioner kan tilføjes uden at påvirke eksisterende logik (Open-Closed Principle (11)).

2.4.2. Designrestriktioner og øvrige krav (+)

FURPS+-modellens "+" dækker over de begrænsninger og rammevilkår, som projektet er underlagt, og for Chroniqo er disse kategoriseret i Tabel 4:

Tabel 4: Oversigt over designrestriktioner og øvrige rammevilkår baseret på FURPS+.

Designrestriktioner og øvrige krav (+)	
Kategori	Beskrivelse og implementering
Tekniske begrænsninger	Systemet udvikles som en webapplikation baseret på moderne web-teknologier (React til frontend og PostgreSQL som den primære datakilde).
Lovmæssige krav (GDPR)	Systemet designes med Privacy by Design ² (12), herunder brugers ret til at blive glemt via <code>ON DELETE CASCADE</code> i databaselaget.
Sikkerhed	Beskyttelse mod uautoriseret adgang håndhæves gennem OAuth 2.0-protokollen og <code>HttpOnly-sessionscookies</code> .
Arkitektonisk restriktion (Chat)	Chat implementeres via short-polling ³ frem for stateful WebSockets for at understøtte den serverløse infrastruktur. Dette er et bevidst trade-off: lavere kompleksitet og driftsomkostninger i MVP-fasen mod en let øget databasebelastning og latens.
Dataintegritet	Indlejrede svar ⁴ begrænses til ét indlejrningsniveau for at forhindre ydeevneproblemer og databaserekursionsfejl ved <code>ON DELETE CASCADE</code> .

² Privacy by Design.

³ Short-polling.

⁴ Nested replies.

2.5. Accepttestspecifikation (Gherkin-scenarier)

For at validere systemets funktionalitet i forhold til de opstillede User Stories er der udarbejdet accepttestscenarier i Gherkin-format. Formatet sikrer en entydig kobling mellem krav og test.

Nedenfor præsenteres to scenarier, der repræsenterer systemets mest Chroniqo-specifikke funktionelle adfærd. De øvrige tre scenarier - dagsforms-synlighed ved privat profil, global søgning i kontekst og global moderation - fremgår af Bilag 1 (8).

2.5.1. Scenarie 1: Registrering af Dagsform (Epic 2 - Must)

Givet: Brugeren er logget ind og befinder sig på forsiden.

Givet: Dagsformsdialogvinduet er aktiveret, da det er første login inden for 24 timer.

Når: Brugeren vælger en dagsform og bekræfter registreringen.

Så: Skal systemet gemme dagsformen og præsentere en støttende systembesked.

2.5.2. Scenarie 2: Anonym interaktion i fællesskaber (Epic 3 + 4 - Must)

Givet: Brugeren er medlem af et fællesskab.

Når: Brugeren opretter et opslag anonymt.

Så: Skal opslaget vises uden brugeridentitet for andre brugere, men være synligt for fællesskabsmoderatorer og systemadministratorer.

2.6. Delkonklusion for kravspecifikation

Med kravspecifikationen på plads tegner sig et klart billede af systemets samlede omfang.

Aktøranalysen identificerede to primære aktører og én understøttende aktør. De 68 User Stories fordelt over fire Epics danner grundlaget for systemets samlede funktionelle omfang, med en MoSCoW-prioritering, der sætter 34 Must-have-krav i centrum for MVP'en. FURPS+-modellen supplerer med 13 målbare kvalitetskrav, der er direkte sporbare til teststrategien i afsnit 7. De to Gherkin-scenarier fastlægger acceptkriterierne for systemets mest systemspecifikke adfærd og sikrer verificerbarhed i praksis.

Kravgrundlaget motiverer de arkitektoniske og designmæssige beslutninger, der behandles i de efterfølgende afsnit.

3. Metode og proces

Projektet anvender en iterativ udviklingsmetode baseret på ASE-modellen (7) kombineret med agile projektstyringsværktøjer. Metodevalget er direkte begrundet i projektets særlige rammebetingelser: et soloforløb med varierende arbejdskapacitet kræver en tilgang, der understøtter løbende prioritering frem for faste sprintforpligtelser. Dette afspejles i valget af procesramme, styringsværktøjer og tidsplan, samt i brugen af generativ AI som fagligt arbejdsredskab.

3.1. Valg af udviklingsmetode

ASE-modellen (jf. Figur 1) er valgt som overordnet procesramme - en iterativ, use case-drevet tilgang, der integrerer elementer fra traditionelle og agile metoder. Den iterative struktur gør det muligt at opdele projektet i håndterbare iterationer, identificere risici tidligt og tilpasse sig løbende, hvilket er afgørende som enkeltmandsudvikler med varierende ressourcer.

Scrum udgør det naturlige sammenligningsgrundlag som den mest udbredte agile metode (13). Scrums sprintforpligtelse og rollekrav er imidlertid designet til teams: for en enkeltmandsudvikler med varierende kapacitet bliver et mislykket sprintmål blot en registreret afvigelse frem for en teaminspiration. Kanban-principperne om visualisering og løbende prioritering er derfor et mere hensigtsmæssigt valg (14). RUP (15) er fravalgt på grund af dets dokumentationsmæssige overhead, der ikke er værdiskabende for et soloprojekt.

3.2. Projektstyring og værktøjer

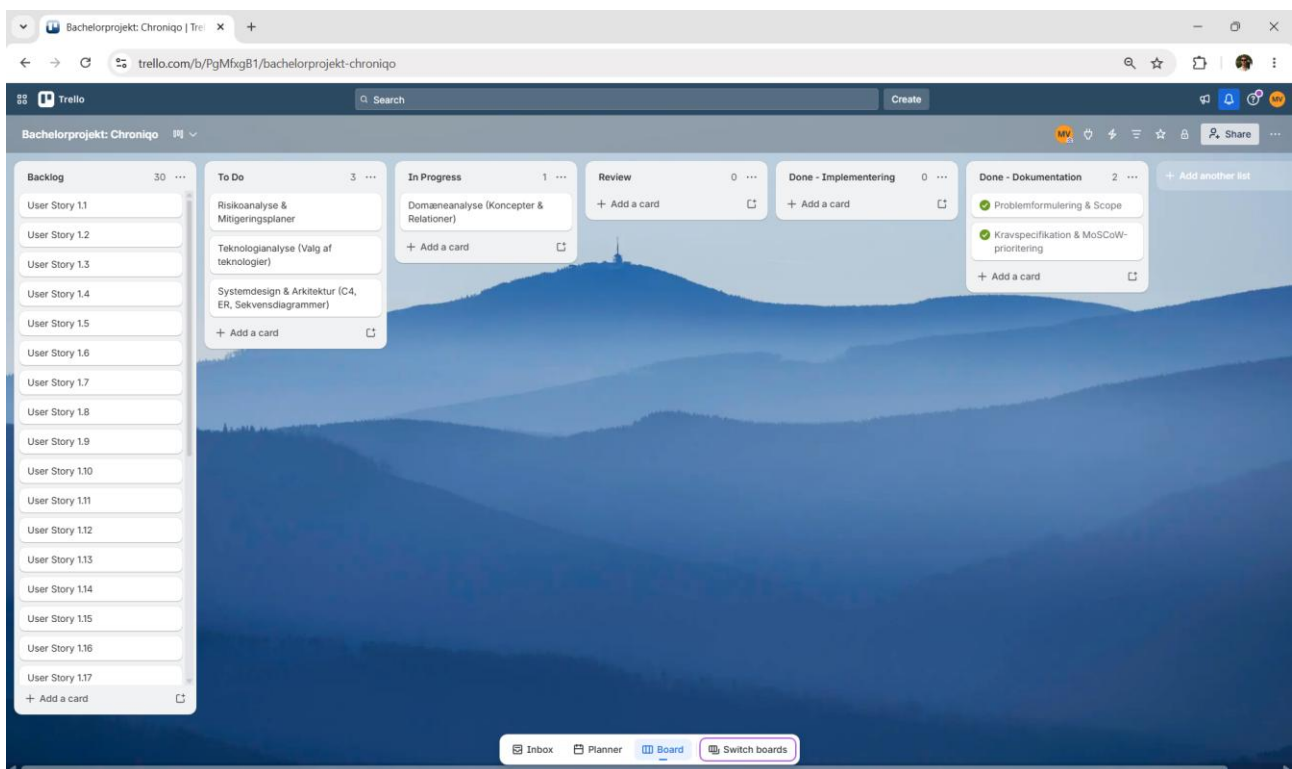
Agile principper kombineres med digitale styringsværktøjer for at skabe overblik og sikre fokus på de mest kritiske opgaver.

3.2.1. Kanban-metoden

Projektets opgaver organiseres efter Kanban-principperne: visualisering af arbejdsflowet, WIP (Work-In-Progress)-begrænsning og konstant fremdrift (14). Det er særligt fordelagtigt som enkeltmandsudvikler, der kræver fleksibel løbende prioritering frem for et fast sprinttempo.

3.2.2. Trello til opgavestyring

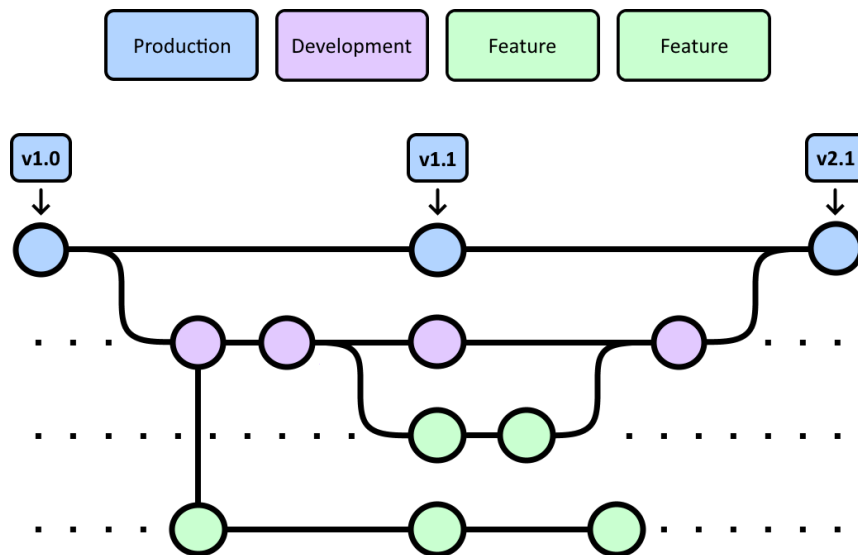
Til opgavestyring benyttes Trello, der implementerer Kanban-flowet med seks lister: *Backlog*, *To Do*, *In Progress*, *Review*, *Done - Implementering* og *Done - Dokumentation*, som vist i Figur 3. Hver kort svarer til en User Story eller en konkret implementeringsopgave og sikrer direkte sporbarhed fra kravspecifikation til udviklingsarbejde.



Figur 3: Projektets Trello-board med opdeling i Backlog, To Do, In Progress, Review, Done - Implementering og Done - Dokumentation. Tavlen visualiserer Kanban-flowet, hvor kortene repræsenterer User Stories og tekniske opgaver. Denne opdeling sikrer et konstant overblik over den iterative udviklingsproces og forhindrer, at for mange opgaver er i gang samtidigt (WIP-begrænsning).

3.2.3. Git til versionsstyring og GitLab som remote repository

GitLab (16) fungerer som remote repository og som en integreret del af udviklingsworkflowet, hvor hver User Story repræsenteres som et issue. Branching-strategien følger feature-branch til development til production-flowet illustreret i Figur 4; ved merge til development fungerer GitLab CI som obligatorisk kvalitetsport, der eksekverer testsuiten, linting og de øvrige jobs, inden kode integreres.



Figur 4: Projektets Git-branching-strategi illustrerer flowet fra feature-branches via development-branchen til production. Nye funktioner isoleres i selvstændige feature-branches, som merges til development efter intern verifikation og succesfuld gennemkørsel af den automatiserede CI/CD-pipeline. Stabile versioner promoveres videre til production, som danner grundlag for deployment. Versionnummereringen (f.eks. v1.0, v1.1) markerer større leverancer.

3.3. Tidsplan

Projektets tidsplan, illustreret i Figur 5, visualiserer overordnede faser og aktivitetsforløb fra uge 5 til uge 22. Planen er bevidst udformet med indlagte buffere for uforudsete tekniske udfordringer og helbredsmæssige perioder og betragtes som dynamisk med løbende justeringer.

Chroniqo Tidsplan																		
Mads Dittmann Villadsen																		
202209869 / AU715368																		
	Jan	Feb				Mar				Apr				Maj				
Uge / Faser	Uge 5	Uge 6	Uge 7	Uge 8	Uge 9	Uge 10	Uge 11	Uge 12	Uge 13	Uge 14	Uge 15	Uge 16	Uge 17	Uge 18	Uge 19	Uge 20	Uge 21	Uge 22
Indledning																		
Kravspecifikation																		
Metode og Proces																		
Analyse																		
Systemdesign og Arkitektur																		
Design & Implementering																		
Test & Debugging																		
Rapportskrivning & Finjustering																		
Generel Buffer																		

Figur 5: Gantt-diagram over projektets tidsplan og faser, der strækker sig fra uge 5 (januar) til uge 22 (maj) 2026. Diagrammet illustrerer det iterative forløb fra kravspecifikation til test og finjustering. For at imødekomme uforudsete helbredsmæssige udfordringer og spidsbelastninger er der indlagt løbende buffere i faserne samt en dedikeret generel buffer-periode mod projektets afslutning.

3.4. Brug af generativ AI

I dette projekt er generativ AI blevet anvendt som et understøttende arbejdsredskab i flere dele af udviklingsprocessen. Brugen er sket inden for rammerne af det tilladte på studiet og har primært haft karakter af faglig sparring og teknisk støtte under udviklingsarbejdet.

Generativ AI er primært blevet brugt som et interaktivt opslagsværktøj og problemløsningsassistent under implementeringsarbejdet. Konkret har det indbefattet hjælp til at undersøge og afklare tekniske spørgsmål, fejlfinde uventede fejl og navigere i dokumentationen for de valgte teknologier og biblioteker.

I en situation som enkeltmandsudvikler uden adgang til sparring fra studiekammerater, har AI-assisterede dialoger fungeret som et refleksivt arbejdsredskab - svarende til at diskutere en teknisk løsning højt - frem for som en direkte kilde til kode, der ukritisk er overført til systemet.

4. Analyse

Analysefasen undersøger projektets domæne, risikoprofil og teknologiske muligheder med henblik på at sikre, at de efterfølgende design- og implementeringsbeslutninger hviler på et systematisk grundlag. De tre delanalyser besvarer tilsammen: hvad skal systemet håndtere, hvad kan gå galt, og hvilke teknologier løser problemet bedst?

4.1. Domæneanalyse

Formålet med domæneanalysen er at adskille domænelogikken fra tekniske implementeringsdetaljer og skabe en konceptuel model over de entiteter og regler, systemet reelt skal håndtere.

4.1.1. Kernekoncepter og forretningsregler

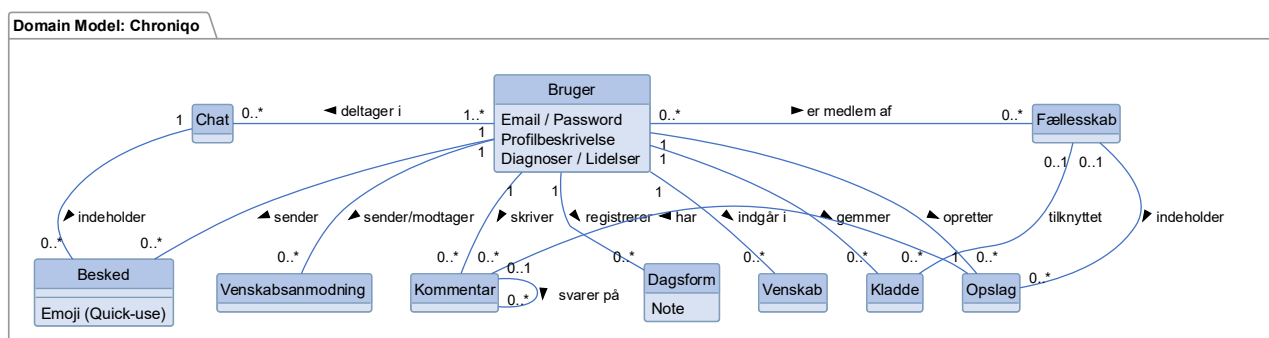
Gennem systematisk analyse af projektets User Stories (8) er ti centrale domænekoncepter identificeret:

Bruger, Fællesskab, Opslag, Kommentar, Dagsform, Chat, Besked, Venskab, Venskabsanmodning og Kladde.

Analysen fastlagde, at Dagsform er en selvstændig entitet med historik - ikke blot en attribut - hvilket er den tekniske forudsætning for en personlig energihistorik over tid. Venskab og Venskabsanmodning er modelleret som selvstændige klasser med egne livscyklusser, da de styrer adgangsrigheder til privat indhold. De stærke hierarkiske afhængigheder - Opslag og Kommentarer har obligatorisk Bruger-ejerskab - nødvendiggør en sletningspolitik med `ON DELETE CASCADE`.

Den vigtigste forretningsregel er maks. én dagsformsregistrering pr. bruger pr. kalenderdag, implementeret som en sammensat database-constraint og begrundelsen for det direkte API-designvalg om `upsert` frem for `create` - en beslutning, der sikrer dataintegritet selv ved samtidige anmodninger.

Disse koncepter og relationer er visualiseret i Figur 6:



Figur 6: Den konceptuelle domæne model for Chroniqo repræsenteret som et UML-klassediagram med ti centrale domænekoncepter og deres indbyrdes relationer. Læsepilene angiver, i hvilken retning domænelogikken fortolkes. Multiplaciteter afspejler konkrete forretningsregler - herunder at én **Dagsform** altid tilhører præcis én **Bruger**, og at en **Kommentar** uadskilleligt er knyttet til et specifikt **Opslag**. Den selvberoende relation på **Kommentar** understøtter ét indlejningsniveau af svar.

4.1.2. Fælles fagsprog (Ubiquitous Language)

Et centralt resultat er etableringen af et fast fælles fagsprog⁵ med 1:1-mapping mellem domænets danske fagtermer og engelske kodekonventioner - f.eks. Dagsform og konventionen `DailyStatus`, og Fællesskab og konventionen `Community` - for at minimere risikoen for misforståelser under udviklingen. Det fulde begrebskatalog fremgår af Bilag 2: Domæneanalyse (17).

⁵ Ubiquitous Language.

4.2. Risikoanalyse

Projektets risikoprofil er analyseret via systematisk identifikation og vurdering af 15 potentielle usikkerheder, visualiseret i en 5x5 risikomatrix i Figur 7. De fire risici med en Impact-score ≥ 15 behandles herunder; de resterende 11 fremgår af Bilag 3: Risikoanalyse (18).

Risikovurderingsmatrix		Sandsynlighed				
		Meget usandsynlig (1)	Usandsynlig (2)	Mulig (3)	Sandsynlig (4)	Meget sandsynlig (5)
Kosekvens	Kritisk (5)	R6: Tab af adgang til repository / versionsstyringsplatform	R7: Dataintegritetsfejl (tab eller inkonsistens i databasedata) R8: Forkert håndtering af personfølsomme data / privacy-fejl	R9: Afhængighed af eksterne tjenester (fx Auth Provider)	R13: Sikkerhedshuller (OAuth 2.0-implementation)	R1: Personlige helbredsudfordringer (Ryg/Fatigue)
	Stor (4)			R2: Scope Creep R3: Estimation bias / tidsoptimisme R4: Fejltolkning eller ændring af krav R5: Utilstrækkelig testdækning	R14: Problemer relateret til systemets deployment	
	Moderat (3)			R10: Performance-problemer (langsomt feed ved mange brugere) R11: Teknisk kompleksitet i anvendte frameworks og biblioteker R12: U hensigtsmæssige arkitektur- eller designbeslutninger		
	Lille (2)			R15: Brugerrelaterede (UX)		
	Ubetydelig (1)					

Figur 7: Risikovurderingsmatrix for Chroniqo-projektet med 15 identificerede risici (R1-R15) fordelt efter vurderet sandsynlighed (x-akse, 1-5) og konsekvens (y-akse, 1-5). Matrixen er farvekodet fra lav risiko (grøn) til høj risiko (rød), og Impact-scoren beregnes som produktet af de to værdier. Fire risici opnår en Impact-score på 15 eller derover og behandles nedenfor; de resterende 11 risici fremgår af Bilag 3 (18).

4.2.1. R1: Personlige helbredsudfordringer (ryg og/eller fatigue)

Den mest kritiske risiko (Impact: 25) med direkte indvirkning på projektets tidsplan og gennemførelse.

Mitigation: *Architectural Simplification* via Next.js-monolittens konsolidering af frontend og backend eliminerer CORS-kompleksitet og reducerer kognitiv belastning; struktureret arbejdstid og ergonomiske hjælpemidler begrænser den fysiske påvirkning.

Contingency: Prioritering af Must-haves og mulighed for online eksamen efter aftale med studienævnet.

4.2.2. R13: Sikkerhedshuller (OAuth 2.0-implementation)

Potentielle sårbarheder i sessionsstyringen (Impact: 20).

Mitigation: Ansvar for kryptering og sessionshåndtering uddelegeres til Auth.js, der håndterer JWT og `HttpOnly`-cookies og minimerer XSS⁶-risici (19).

Contingency: Isolering af sårbarhed og tvungen adgangskodenulstilling.

4.2.3. R14: Problemer relateret til systemets deployment

Risiko for fejl under deployment til produktionsmiljøet (Impact: 16).

Mitigation: Detaljeret deployment-dokumentation og staging-miljø.

Contingency: Rollback til stabil version.

4.2.4. R9: Afhængighed af eksterne tjenester

Risiko for utilgængeligt tredjeparts-API som Google OAuth (Impact: 15).

Mitigation: Fallback til manuelt credentials-login.

Contingency: Aktiv kommunikation til brugere om alternative loginmetoder.

⁶ XSS (Cross-Site Scripting).

4.3. Teknologianalyse

Teknologianalysen forbinder kravene fra afsnit 2 og risikoanalysen fra afsnit 4.2 med den endelige arkitektur med henblik på systematiske teknologivalg styret af: høj Developer Experience (DX) for en enkeltmandsudvikler, stærk datasikkerhed, fuld typesikkerhed og omkostningsfri serverless infrastruktur.

4.3.1. Teknologivalg

Baseret på en struktureret vurdering af alternativer - herunder Python/Django, Node.js/Express og NoSQL-databaser som MongoDB og Firebase - er teknologistakken i Tabel 5 valgt. Den fulde begrundelse med alternativer og trade-offs fremgår af Bilag 4: Teknologianalyse (20).

Tabel 5: Oversigt over Chroniqos teknologistak fordelt på kategori og komponent. Valget af teknologier er styret af fire arkitektoniske principper: Developer Experience, Skalerbarhed, Datasikkerhed og Integritet samt Vedligeholdelse og Dokumentation. Den fulde begrundelse for hvert valg med alternativer og tradeoffs fremgår af Bilag 4: Teknologianalyse (20).

Teknologivalg for Chroniqo-projektet	
Kategori / Komponent	Teknologivalg
Fullstack framework	Next.js (App Router) med TypeScript, hostet serverless på Vercel (21-23).
Styling og responsivitet	Tailwind CSS (24).
Databaseteknologi	PostgreSQL (Relationel) med Prisma ORM, hostet på Neon.tech (25-27).
Caching og rate limiting	Serverless Redis via Upstash (28, 29).
Autentificering og sikkerhed	Auth.js (NextAuth v5), der abstraherer OAuth 2.0 PKCE-flowet ⁷ og håndterer stateless JWT-sessioner via sikre HttpOnly-cookies (19, 30-32).
Realtidskommunikation (chat)	Short-polling (via SWR ⁸) frem for stateful WebSockets, som et bevidst arkitektonisk MVP-valg (33).
AI-integration	Google Gemini API (Gemini 2.5 Flash Lite) implementeret sikkert server-side (34).
Infrastruktur	Docker til container-versionering af produktionsimaget via GitLab CI/CD-pipeline; Managed Cloud (Vercel) til applikationshosting (16, 22, 35).
Testframework og kvalitetssikring	Jest via next/jest med jest-mock-extended til typesikker Prisma-mocking, eksekveret automatisk i GitLab CI/CD-pipeline (16, 36, 37).

⁷ Proof Key for Code Exchange (PKCE).

⁸ SWR (Stale-While-Revalidate).

4.3.2. Arkitektoniske overvejelser og risikohåndtering

Tre strategiske greb forbinder teknologivalgene med projektets specifikke udfordringer.

4.3.2.1. Architectural Simplification (R1 (18))

Next.js-monolitten konsoliderer frontend og backend med end-to-end typesikkerhed via Prisma og én samlet CI/CD-pipeline, der markant reducerer kognitiv belastning og CORS-kompleksitet for enkeltmandsudviklingen.

4.3.2.2. Delegeret sikkerhed (R13 (18))

Auth.js abstraherer OAuth-kryptering og sessioner; sliding-window rate limiting i Edge Middleware⁹ via Upstash Redis afviser brute-force-trafik proaktivt, inden den rammer databasen.

4.3.2.3. Privacy by Design (US3.19 (8))

Route Handlers filtrerer PII¹⁰ - navne, e-mailadresser, telefonnumre og CPR-numre - server-side via regex, inden prompts afsendes til Gemini API; chathistorik begrænses til seneste samtale, og ON DELETE CASCADE sikrer GDPR-compliance ved kontosletning.

Den fulde analyse fremgår af Bilag 4 (20).

⁹ Edge Network.

¹⁰ PII (Personally Identifiable Information).

4.4. Delkonklusion for analyse

De tre delanalyser peger på den samme arkitektoniske konklusion. Domæneanalysen identificerede ti kernekoncepter og fastlagde `upsert`-reglen for Dagsformen som en kritisk forretningsregel, der stiller direkte krav til databasedesignet. Risikoanalysen fremhævede R1 og R13 som risici med direkte arkitektonisk virkning. Teknologianalysen dokumenterede, at en Next.js-monolit med delegeret sikkerhed og serverless infrastruktur bedst imødekommer samtlige analysers krav på én gang.

Analysefasens samlede fund danner det direkte fundament for systemdesignet i afsnit 5.

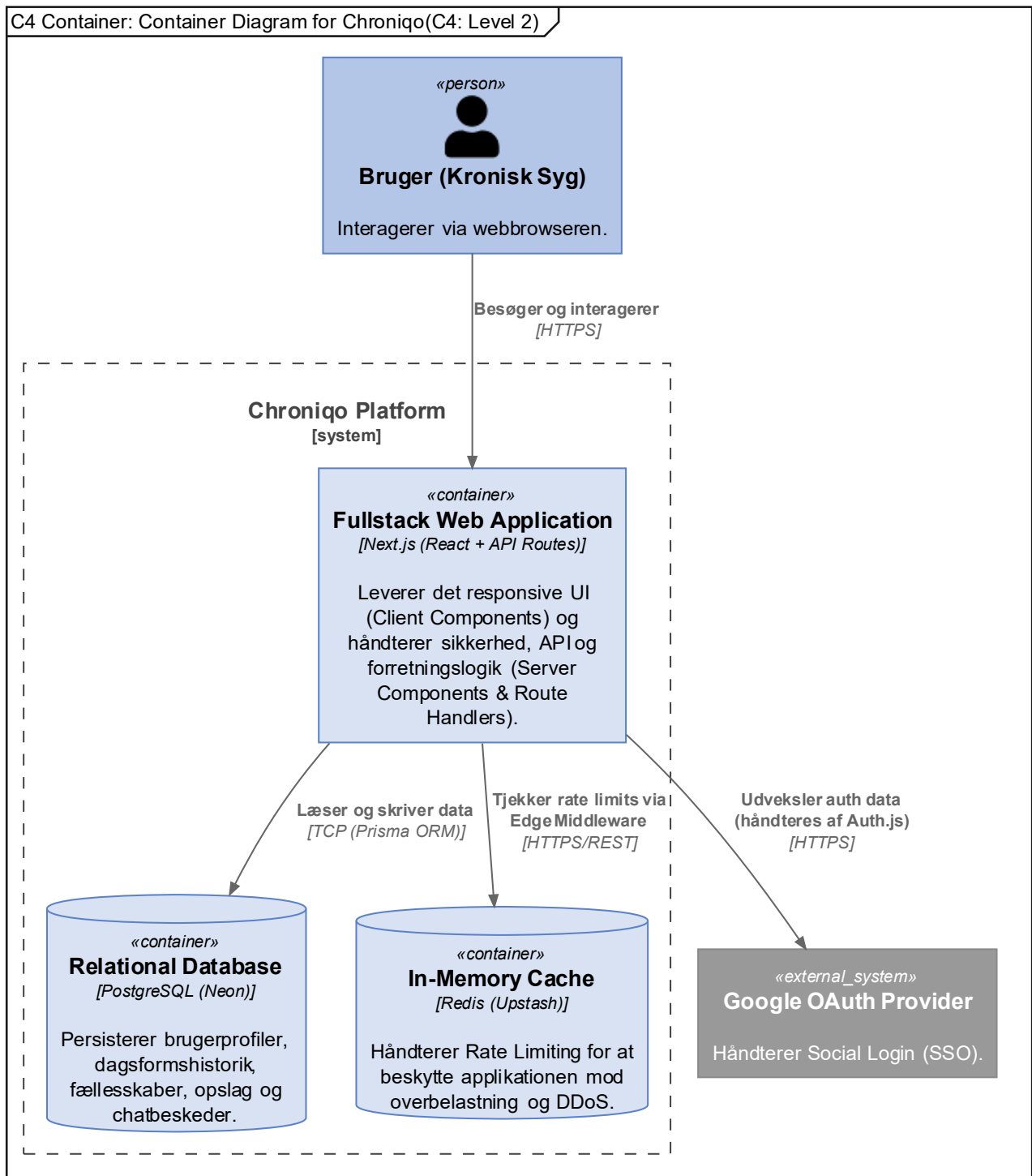
5. Systemdesign og arkitektur

Hvor de foregående afsnit har analyseret krav, risici og domæne, tager arkitekturarbejdet fat på den tekniske realisering af Chroniqo. Systemet præsenteres på tre niveauer: den overordnede opbygning, databasedesignet og applikationsmodellens lagdeling.

For en fuldstændig gennemgang af alle designbeslutninger - herunder samtlige C4-niveauer, Data Dictionary, API-kontrakter og de resterende sekvensdiagrammer - henvises til Bilag 5: Systemdesign & Arkitektur (38).

5.1. Systemarkitektur (C4-modellen)

For at anskueliggøre systemets overordnede opbygning og integrationer anvendes C4-modellen (39). Container-diagrammet (Level 2) i Figur 8 illustrerer de uafhængigt deployerbare enheder og deres kommunikation:



Figur 8: C4 Container diagram (Level 2) for Chroniqo. Viser konsolideringen af klient- og backend-logik i én Next.js fullstack-webapplikation, understøttet af PostgreSQL via Prisma ORM, Upstash Redis til rate limiting i Edge-netværket og Google OAuth Provider til autentificering (22, 25, 29, 30).

Der er truffet en strategisk beslutning om at konsolidere klient- og backend-logik i ét Next.js-framework frem for en adskilt frontend/backend-arkitektur (23). Dette reducerer kognitiv belastning og administrativ kompleksitet markant og afbøder derved projektets primære risiko direkte (jf. R1: Helbredsudfordringer (18)). Sikkerhedsmæssigt håndterer Auth.js (30) autentificering og etablerer `HttpOnly`-cookies (19) uden at eksponere tokens for klienten, mens Upstash Redis (29) bruges til rate limiting i Next.js Edge Middleware (40) for at afvise ondsindet trafik, inden det rammer applikationens kerne. Deployment sker via managed cloud-tjenester - Vercel, Neon.tech og Upstash (22, 27, 29) - i en NoOps¹¹-model uden driftsomkostninger. Det interne komponentdiagram (Level 3), der viser anmodningens rejse fra Client Components via Edge Middleware til Service-lag og Prisma, samt en fuld deployment-oversigt fremgår af Bilag 5, afsnit 2 (38).

¹¹ NoOps (No Operations).

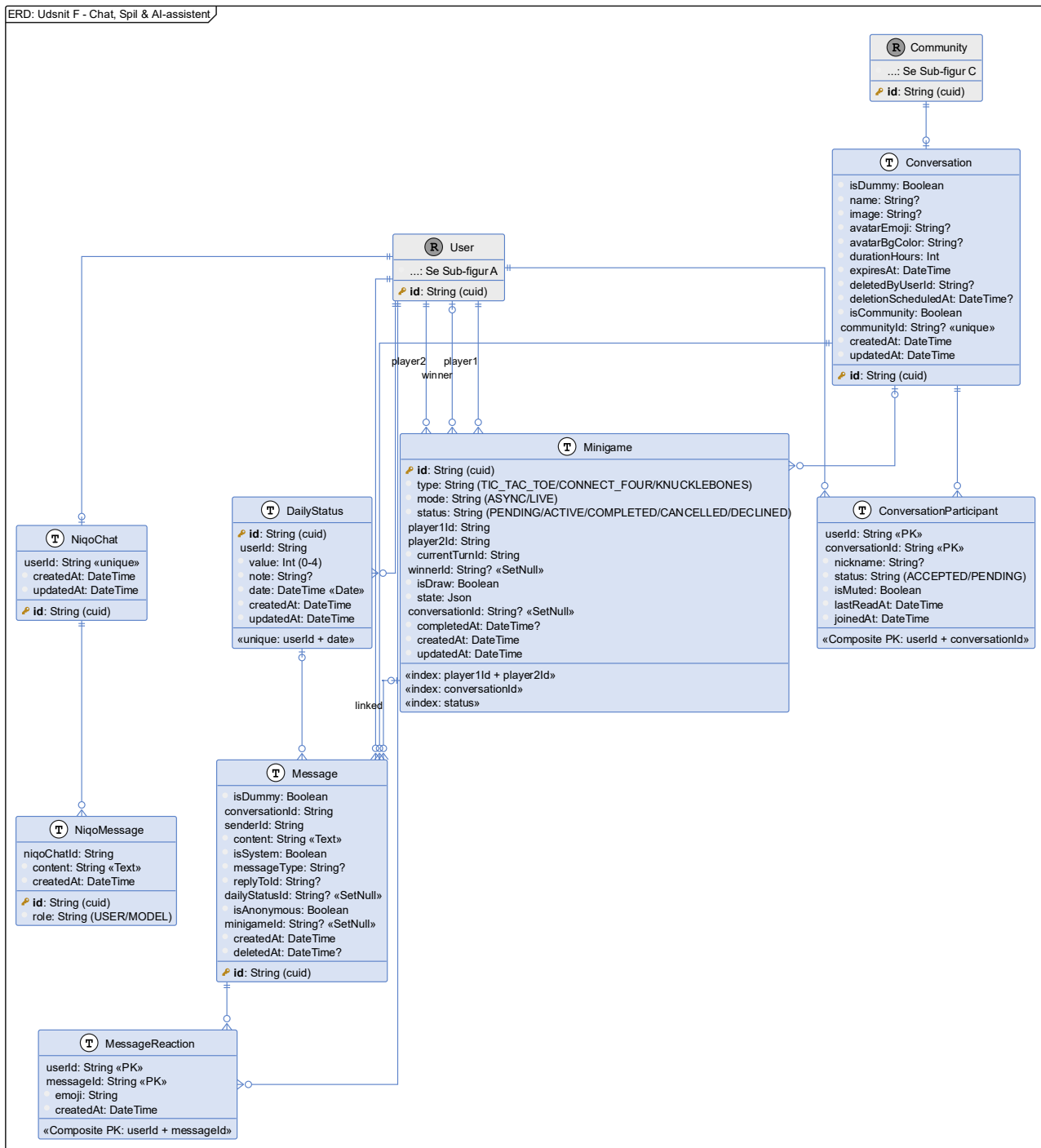
5.2. Databasedesign

Den konceptuelle domænemodel er transformeret til 40 relationelle tabeller i PostgreSQL (26) via Prisma Object-Relational Mapper (ORM) (25). To principper gennemsyrrer hele skemaet: `cuid`¹²-baserede primærnøgler gør ID'er uforudsigelige og forhindrer ID-enumerationsangreb (41), og GDPR-kravet om retten til at blive glemt (42) er implementeret via `ON DELETE CASCADE` på al personhenførbare data tilknyttet `User`. Den fulde ER-model med alle 40 modeller og feltspecifikationer fremgår af Bilag 5, Figur 4a-4g (38).

Det domæneudsnit, der er tættest koblet til platformens adaptive kernefunktionalitet, fremgår af Figur 9:

¹² `cuid` (Collision-resistant Unique Identifier).

ERD: Udsnit F - Chat, Spil & AI-assistent



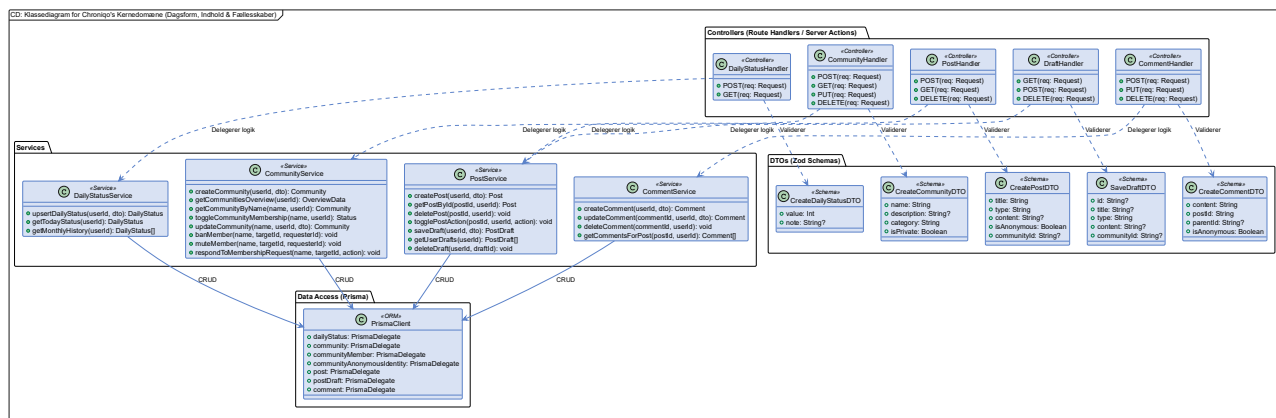
Figur 9: ER-diagram (Sub-figur F) for kommunikations- og kernetdomænet. Viser *DailyStatus* med Composite Unique Constraint [userId, date], *Conversation* med udløbsmekanisme (expiresAt), *Message* med blød sletning og anonymitetsstøtte, *MessageReaction* med én reaktion pr. bruger pr. besked, *MiniGame* (understøtter TIC_TAC_TOE, CONNECT_FOUR, KNUCKLEBONES) samt *NiqoChat* og *NiqoMessage* til AI-assistentens beskedhistorik (25).

`DailyStatus` håndhæver forretningsreglen om maks. én dagsformsregistrering pr. bruger pr. kalenderdag via Composite Unique Constraint¹³ på `[userId, date]`. `Conversation` implementerer platformens tidsbegrænsede chatdesign via `expiresAt`. `CommunityAnonymousIdentity` - modelleret i Sub-figur C i Bilag 5 (38) - persisterer én stabil, autogenereret anonym identitet pr. bruger pr. fællesskab, der er det tekniske fundament for platformens anonymitetsfunktion.

¹³ Composite Unique Constraint.

5.3. Applikationsmodel (softwaredesign)

Next.js-applikationen er struktureret efter et flerlagret arkitekturmønster¹⁴ (43), som illustreret i Figur 10:



Figur 10: Design-klassediagram for systemets kernerområde. Viser lagdelingen fra Controllers (Route Handlers) til Data Access via dedikerede Services. Pilnotationen skelner mellem permanente afhængigheder (fuldt optrukne pile) og flygtige dataoverførsler via DTO'er (stiplede pile) (44).

Route Handlers fungerer udelukkende som Controllers, der validerer input via Zod-baserede DTO'er¹⁵ (44, 45), mens al forretningslogik er isoleret i Services og databaseadgang håndteres gennem Prisma (25). DTO-laget modvirker over-posting ved at definere nøjagtigt hvilke felter der må modtages fra klienten - felter som `role` accepteres simpelthen ikke fra request-bodyen, men hentes udelukkende fra den server-side session. Et tilsvarende diagram for kommunikationsdomænet (ChatService, NiqoService og MinigameService) fremgår af Bilag 5, Figur 7 (38).

¹⁴ Layered Architecture.

¹⁵ DTO (Data Transfer Object).

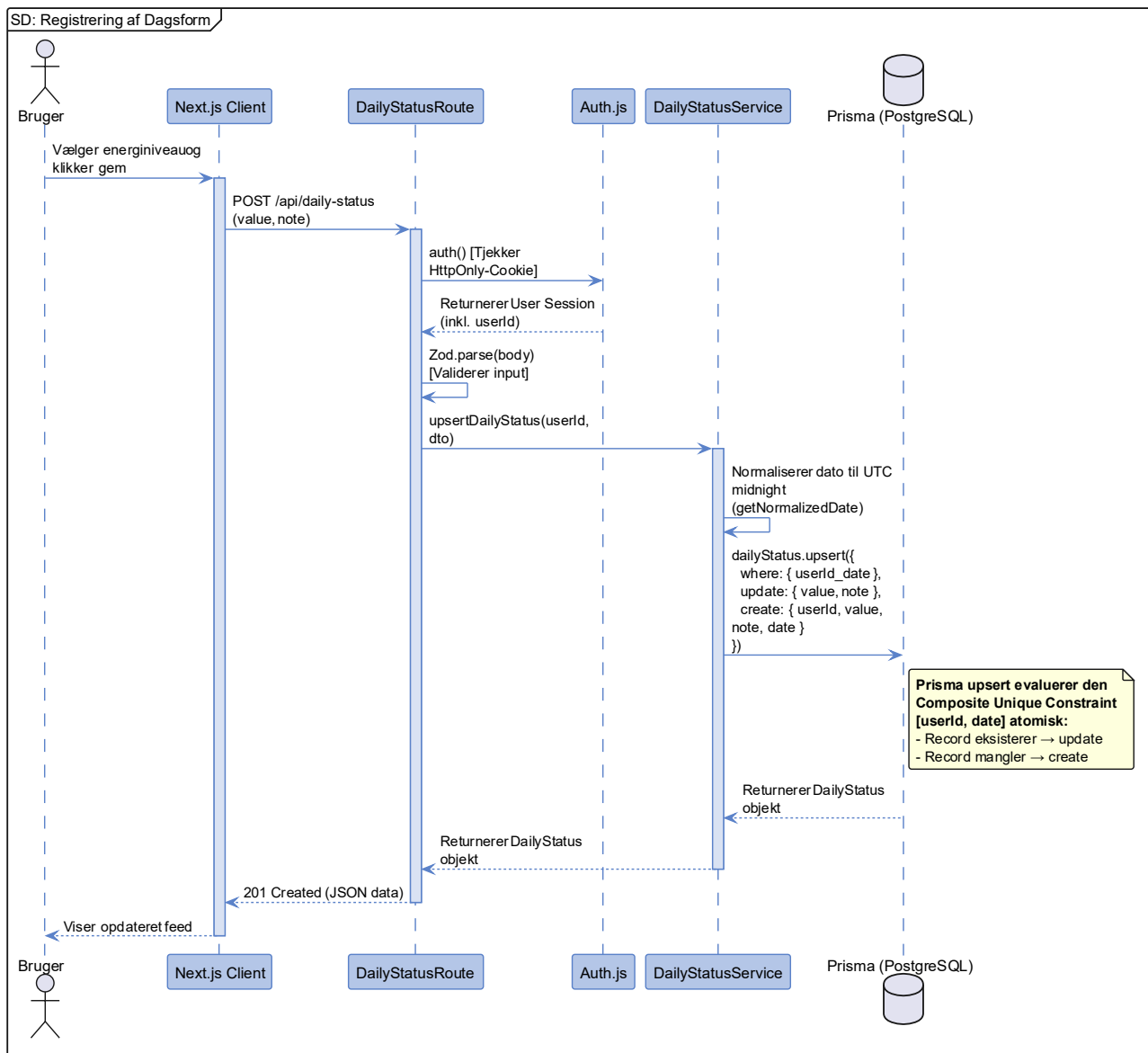
5.4. Dynamisk adfærd (sekvensdiagrammer)

Systemets dynamiske adfærd er verificeret for det arkitektonisk mest centrale flow. Rate limiting-flowet, herunder Edge Middleware-afvisning og EVALSHA-kommandoen mod Upstash Redis, er modelleret og dokumenteret med kodeudsnit i afsnit 6.3.3. De resterende ni sekvensdiagrammer - herunder brugeroprettelse, OAuth PKCE-flow, anonymt opslag, optimistic UI¹⁶ og AI-støttebot med PII-filtrering - fremgår af Bilag 5, afsnit 5 (38).

5.4.1. Registrering af Dagsform (US2.1)

Dagsformen er systemets omdrejningspunkt. Sekvensdiagrammet i Figur 11 viser, hvordan Auth.js validerer sessionscooken, inden Route Handleren delegerer til Service-laget, som via en enkelt atomisk Prisma `upsert`-operation evaluerer Composite Unique Constraint `[userId, date]` og afgør, om dagsformen skal oprettes eller opdateres - uden separat `findFirst`-forespørgsel og uden risiko for race conditions (25).

¹⁶ Optimistic UI.



Figur 11: Sekvensdiagram for Registrering af Dagsform (US2.1). Illustrerer Auth.js-sessionstjek via `HttpOnly`-cookie (19) og den atomiske Prisma `upsert`-operation, der håndhæver forretningsreglen om maks. én dagsformsregistrering pr. bruger pr. kalenderdag (25).

5.5. API-design (grænseflader)

Den tekniske kontrakt mellem Next.js Client Components og systemets Route Handlers er designet efter RESTful-principper (46). API'et kommunikerer via JSON-payloads og benytter standardiserede HTTP-statuskoder (som 201 Created, 401 Unauthorized og 429 Too Many Requests) (47) for at sikre en forudsigelig og skalerbar fejlhåndtering på klienten. Al sessionshåndtering varetages automatisk af Auth.js, hvorfor det ikke er nødvendigt manuelt at medsende Authorization-headers. En komplet specifikation af systemets centrale REST-endpoints, herunder request/response-kontrakter og sikkerhedsparametre, fremgår af Bilag 5, afsnit 6 (38).

5.6. Delkonklusion for systemdesign og arkitektur

Med systemets tekniske byggeplan nu kortlagt på tværs af fire perspektiver tegner sig et klart billede af de arkitektoniske valg, der bærer Chroniqo.

Den overordnede arkitektur er bevidst konsolideret i en Next.js monolitisk fullstack-løsning med en lagdelt intern struktur - en beslutning, der reducerer kognitiv belastning og afbøder risiko R1 direkte. Databasedesignet realiserer domænemodellen i 40 relationelle tabeller med strenge constraints, herunder `CommunityAnonymousIdentity` som det tekniske fundament for platformens anonymitetsfunktion.

Sekvensdiagrammet har verificeret, at det statiske design fungerer for systemets absolut mest centrale flow: den atomiske dagsformsregistrering, der udgør kernen i platformens adaptive design.

Med arkitekturen fastlagt som baseline udgør dette afsnit det direkte fundament for afsnit 6, der beskriver, hvordan disse beslutninger er realiseret i den endelige implementering.

6. Design og implementering

Arkitekturen fra afsnit 5 er en intention; dette afsnit er realiseringen. Fra designsystemets farvekodede dagsformsindikatorer over Prisma-skemaets håndhævede forretningsregler til den fuldt automatiserede CI/CD-pipeline omsættes hvert arkitektonisk princip til konkrete implementeringsbeslutninger med eksplicitte trade-offs.

Den fulde tekniske dokumentation med kodeudsnit og konfigurationsfiler fremgår af Bilag 6: Design & Implementering (48); den komplette kildekode af Bilag 9: Kildekode (49).

6.1. UI/UX-design og visuel identitet

Brugergrænsefladen er udviklet med et kompromisløst fokus på at minimere kognitiv belastning for platformens primære målgruppe - brugere med kronisk sygdom og varierende dagsform - i overensstemmelse med WCAG-standarderne (50) (jf. krav UR3 (8)).

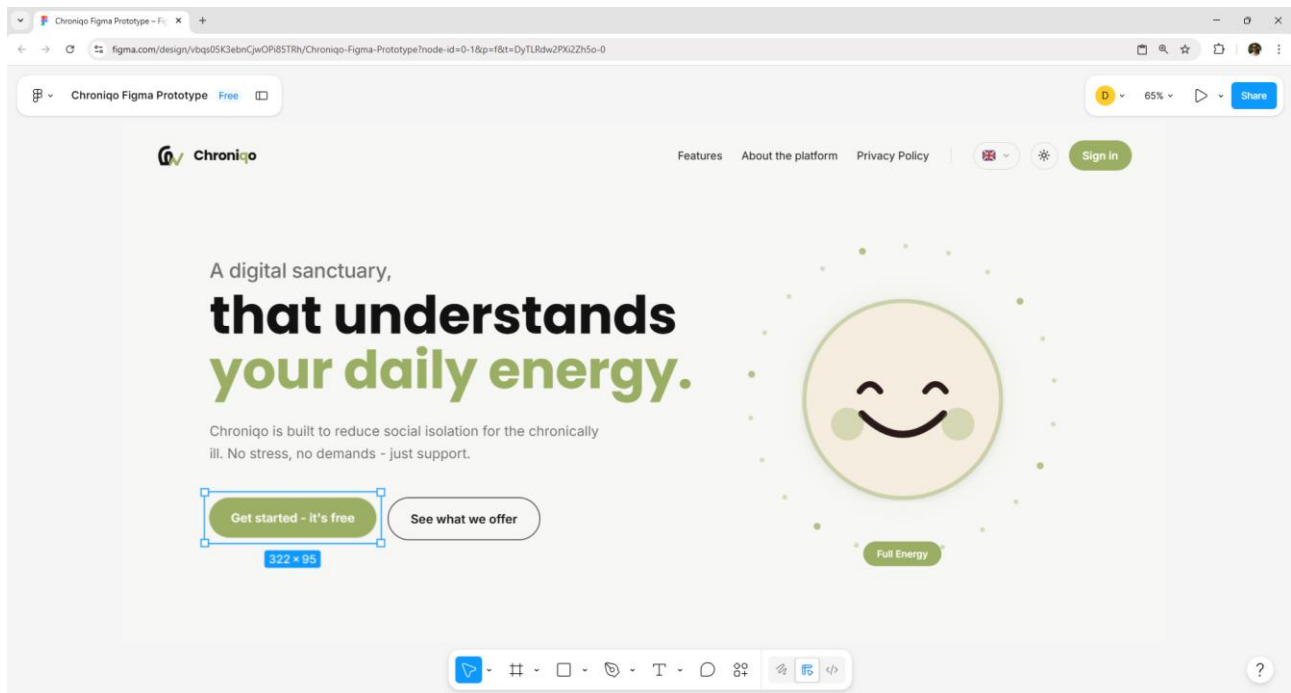
6.1.1. Designsystem og tilgængelighed (Accessibility)

Som det fremgår af Figur 12, er designets typografi og farvepalet valgt med kognitiv skånsomhed som bærende princip. Typografien er opdelt i Poppins (51) til overskrifter for dens åbne, geometriske former og Inter (52) til brødtekst for dens anerkendte digitale læsbarhed. For at reducere kontrastinduceret stress benytter *Light Mode* en varm off-white (#F7F7F5) frem for kridthvid (#FFFFFF), og *Dark Mode* en dyb grå-sort (#121212). Platformens brandfarve, Vibrant Care (#E65C69), signalerer omsorg uden at fremstå klinisk alarmerende og anvendes udelukkende på Call-to-Action-elementer for at sikre intuitiv navigation uden visuel støj.



Figur 12: Platformens dual-theme farvepalet i henholdsvis *Light Mode* (varm off-white #F7F7F5) og *Dark Mode* (dyb grå-sort #121212) samt den primære brandfarve Vibrant Care (#E65C69). Den varme off-white er valgt frem for kridthvid (#FFFFFF) for at reducere kontraststress hos lysfølsomme brugere. Brandfarven anvendes udelukkende til Call-to-Action-elementer for at sikre klar, intuitiv navigation uden visuel støj.

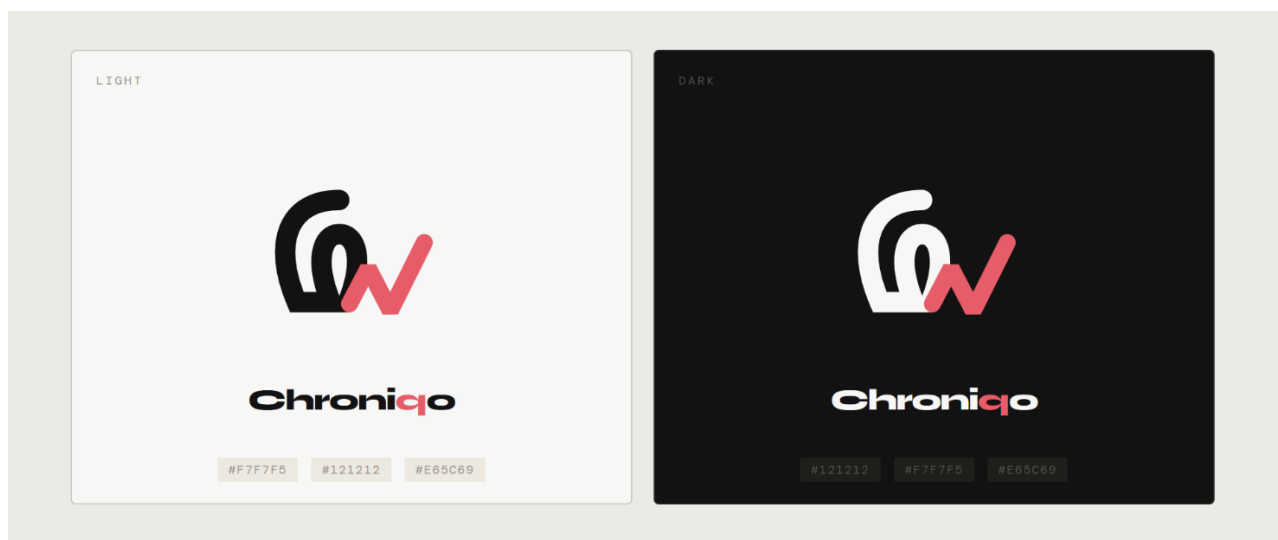
Designsystemet er implementeret via CSS custom properties i `globals.css` og Tailwind CSS v4's `@theme inline`-mekanisme, der eksponerer tokens som Tailwind-utilities og sikrer automatisk temaskift på tværs af hele komponenttræet uden individuelle komponentændringer. Inden kodningen påbegyndtes, blev brugerflows valideret via en interaktiv Figma-prototype, der etablerede de designtokens, der mappede direkte til Tailwind-klasser og minimerede den semantiske afstand mellem designartefakt og implementering. Et udsnit af prototypen fremgår af Figur 13.



Figur 13: Udsnit af Chroniqos interaktive Figma-prototype, der viser landingssiden i en tidlig designfase. Hero-sektionen præsenterer platformens kernekoncept med en smiley-emoji i farven Full Energy som visuel anker. De to Call-to-Action-knapper - "Get started - it's free" og "See what we offer" - er placeret i fokus. Prototypen er udformet på engelsk og fungerede som grundlag for validering af navigationshierarki og brugerflows inden implementeringen påbegyndtes.

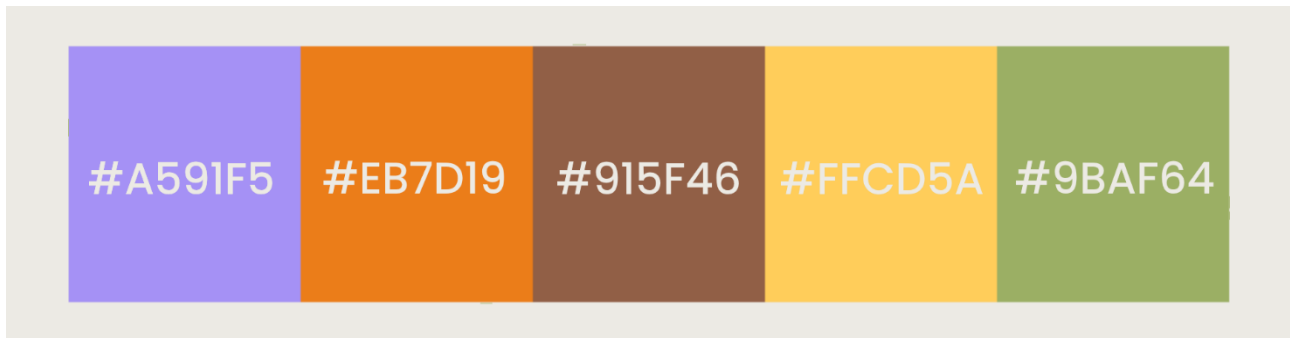
6.1.2. Logo og visuelt udtryk

Chroniqos logo, illustreret i Figur 14, er designet som et monogram, der sammensætter platformens initialer ("C" og "Q") med en EKG-bølge, som symboliserer samspillet mellem sygdomsindsigt og social forbundethed. EKG-bølgen er fremhævet i brandfarven for at skabe dynamik i det minimalistiske udtryk.



Figur 14: Chroniqos monogramlogo i henholdsvis Light Mode og Dark Mode. Logoet sammensætter platformens initialer "C" og "Q" med en EKG-bølge, der symboliserer samspillet mellem sygdomsindsigt og social forbundethed. EKG-bølgen er fremhævet i brandfarven Vibrant Care (#E65C69) for at skabe dynamik i det minimalistiske udtryk og drage blikket mod logoets centrale budskab om liv og omsorg.

Dagsformen er visuelt kodet via fem unikke pastelfarver fra lilla (#A591F5 - Helt Udmattet) og orange (#EB7D19 - Lavt Overskud) over brun (#915F46 - Nogenlunde) til gul (#FFCD5A - Gode Perioder) og grøn (#9BAF64 - Fuldt Overskud), illustreret i Figur 15. Farverne er bevidst valgt i dæmpede nuancer for at minimere visuel støj.



Figur 15: Dagsformens fem farvekodede energiniveauer fra Helt Udmattet (lilla #A591F5) over Lavt Overskud (orange #EB7D19), Nogenlunde (brun #915F46) og Gode Perioder (gul #FFCD5A) til Fuldt Overskud (grøn #9BAF64). Farverne er bevidst valgt i dæmpede pastelnuancer for at minimere visuel støj, så brugergrænsefladen forbliver rolig og ikke-overvældende, selv når farverne benyttes til at farvekode opslag og profilvisninger.

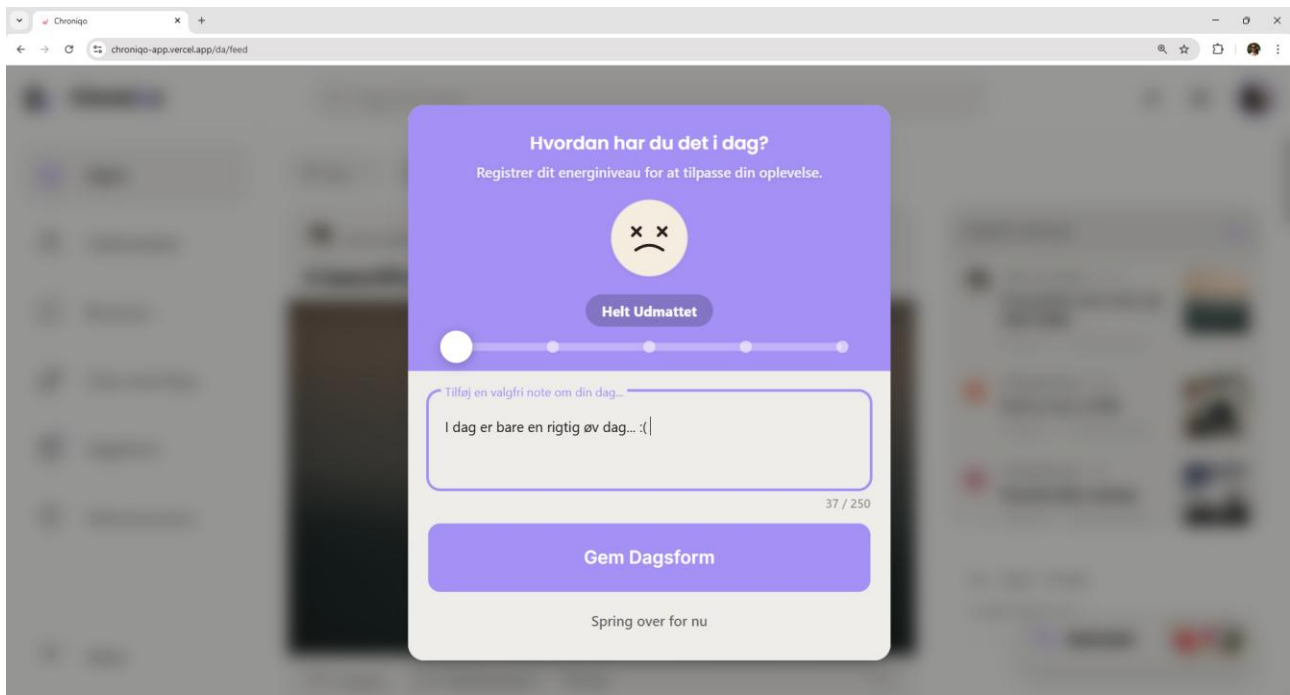
6.1.3. Dagsformsindikatoren i praksis

Dagsformen udgør systemets omdrejningspunkt og fungerer som den primære asynkrone kommunikationsform mellem brugerne (jf. krav US2.1 (8)).

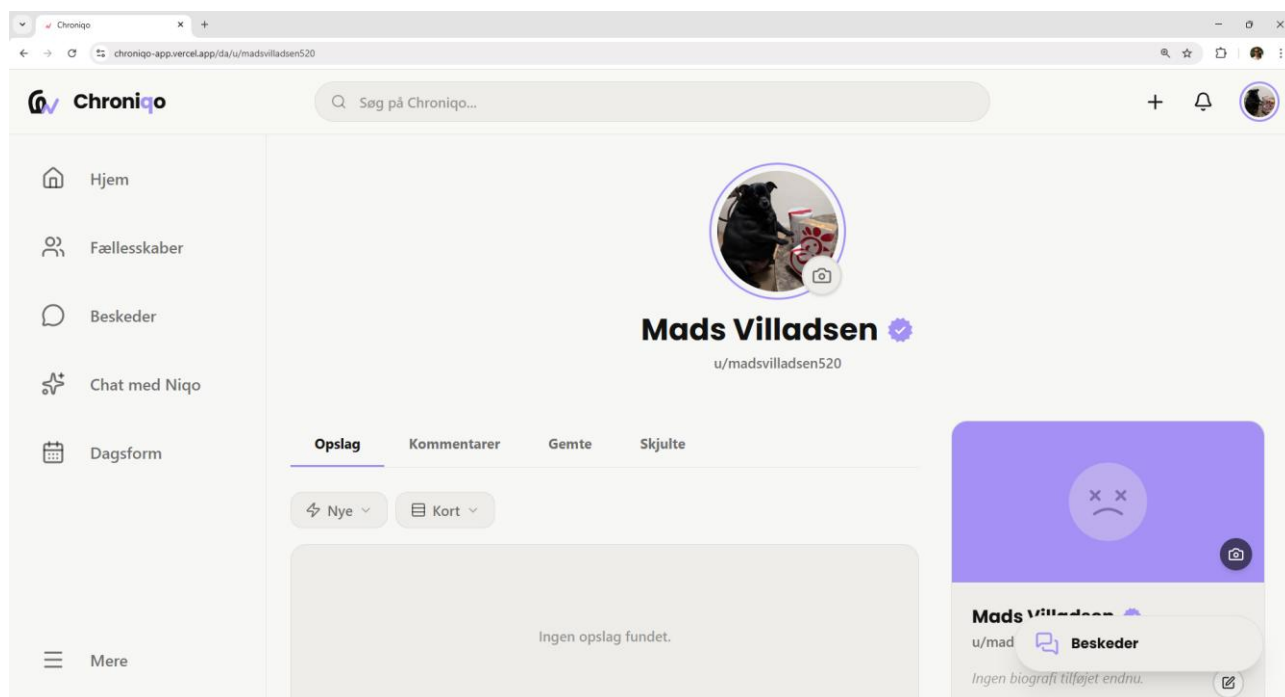
For at sikre, at registreringen ikke føles som en byrde, er grænsefladen designet til minimal friktion - brugeren kan desuden springe registreringen over på dage, hvor interaktion føles uoverskueligt.

Når en dagsform er registreret, integreres farvekoden subtilt i brugerens profil. Farven vises som en ring om profilbilledet, samt som en diskret baggrundsnuance på profilens cover-art, såfremt brugeren ikke selv har tilpasset sit cover-art. Dette kommunikerer energiniveauet helt uden brug af tekst.

Registreringsgrænsefladen og den efterfølgende profilvisning fremgår af henholdsvis Figur 16 og Figur 17:



Figur 16: Dagsformsinterceptoren, der præsenteres for brugeren ved første besøg i en ny session. Grænsefladen er farvet i lilla - den farve, der svarer til energiniveauet Helt Udmattet - og viser et tydeligt emoji-ikon med nedadgående mund som umiddelbar visuel repræsentation af udmattelse. En kurveformet slider med fem positioner repræsenterer spektret fra Helt Udmattet til Fuldt Overskud, hvor den aktive markør befinder sig yderst til venstre. Et valgfrit tekstfelt tillader brugeren at tilføje en personlig note til registreringen. Knappen "Gem Dagsform" gemmer valget, mens linket "Spring over for nu" giver brugeren mulighed for at afstå fra registrering på dage, hvor interaktion føles overvældende.



Figur 17: Profilsiden for en bruger, der har registreret dagsformen Helt Udmattet. Profilbilledet er omgivet af en tydelig lilla ring - den farve, der er tildelt det laveste energiniveau - der fungerer som et passivt, ikke-verbalt signal til øvrige brugere på platformen. Ringen er synlig både på profilsiden og i forbindelse med brugerens opslag i feedet, så andre med ét øjekast kan aflæse personens aktuelle energiniveau uden nogen form for tekstlig forklaring.

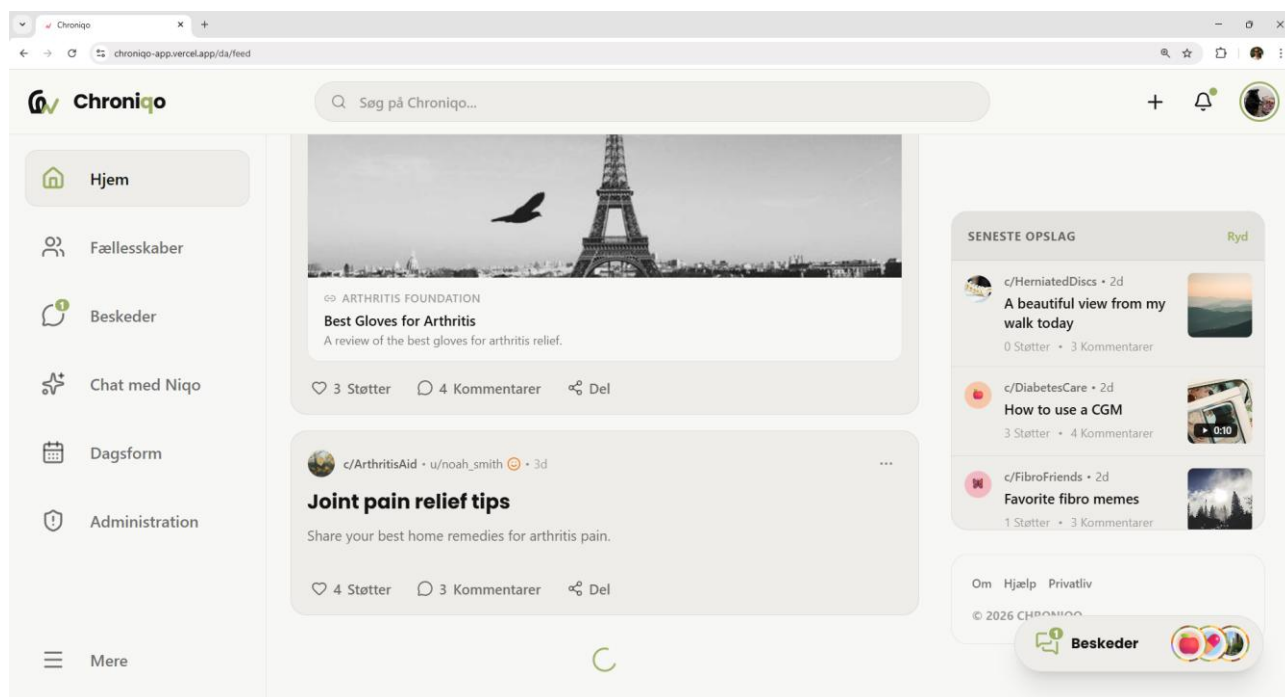
6.1.4. Det adaptive UI (Low Energy Mode)

Hvor dagsformsindikatoren kommunikerer brugerens tilstand udadtil, er *Low Energy Mode* mekanismen, der beskytter brugeren indadtil. Dette afsnit besvarer projektets centrale problemformulering om teknologisk adaptation til individets aktuelle dagsform (jf. krav US2.13 (8)).

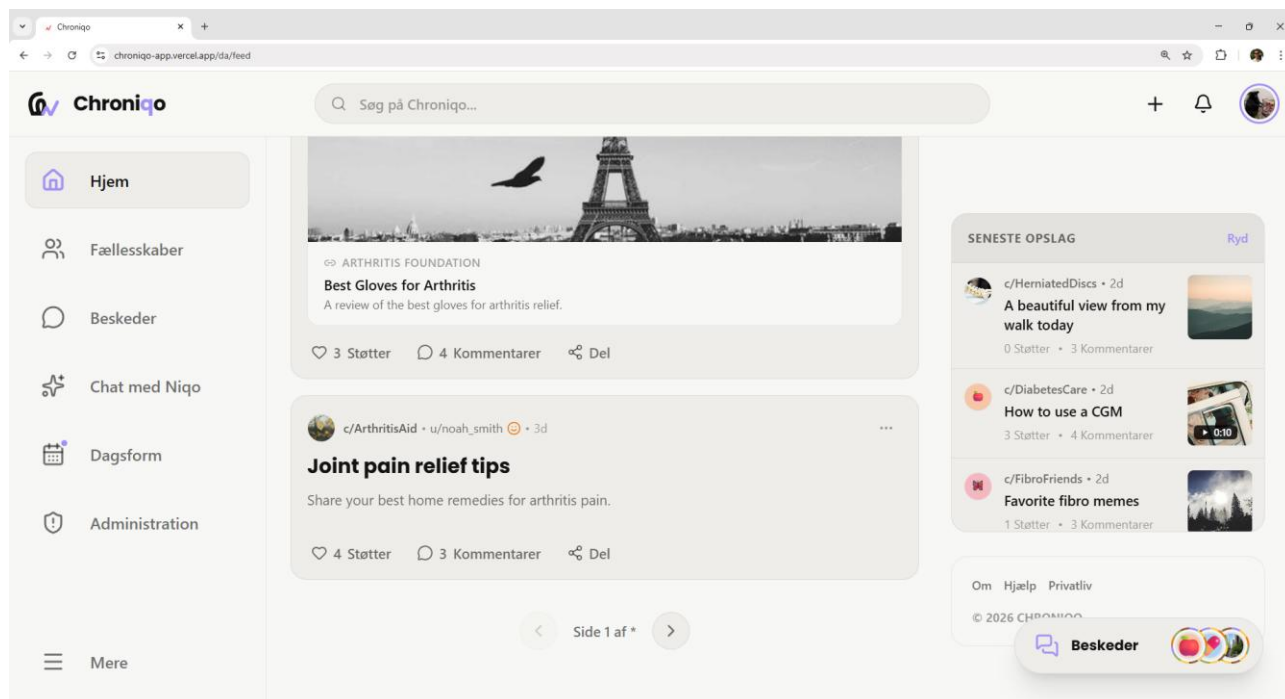
Når en bruger registrerer et lavt energiniveau (niveau 0 eller 1, nulindekseret), iværksætter systemet automatisk to prædefinerede UI-reduktioner: notifikationsbadges skjules for at fjerne den iboende opfordring til at handle straks, og uendeligt scroll erstattes af paginering for at modvirke doomscrolling (53) (jf. krav US2.13 (8)).

Den tekniske mekanisme er centraliseret i `DailyStatusContext`, der distribuerer `currentValue` - dagsformsniveauet på en skala fra 0 til 4 - til hele applikationstræet via React Context API (54). Alle adaptive komponenter tilgår denne ene tilstandsvariabel og renderer betinget: er `currentValue` 0 eller 1, aktiveres *Low Energy Mode* uden yderligere API-kald. Straks efter registrering muteres den globale CSS-variabel `--brand` synkront med dagsformens farve, og `markRegistered()` invaliderer samtlige `/api/conversations-cache` indgange via SWRs `globalMutate`, så åbne chatvisninger i den aktuelle session opdateres øjeblikkeligt. Den fulde komponentimplementering er dokumenteret i Bilag 6, afsnit 2.2-2.3 (48).

Forskellen på feedet i standardtilstand og *Low Energy Mode* fremgår af Figur 18 og Figur 19. En komplet visuel gennemgang af alle platformens sider fremgår af Bilag 8: UI-Dokumentation (55).



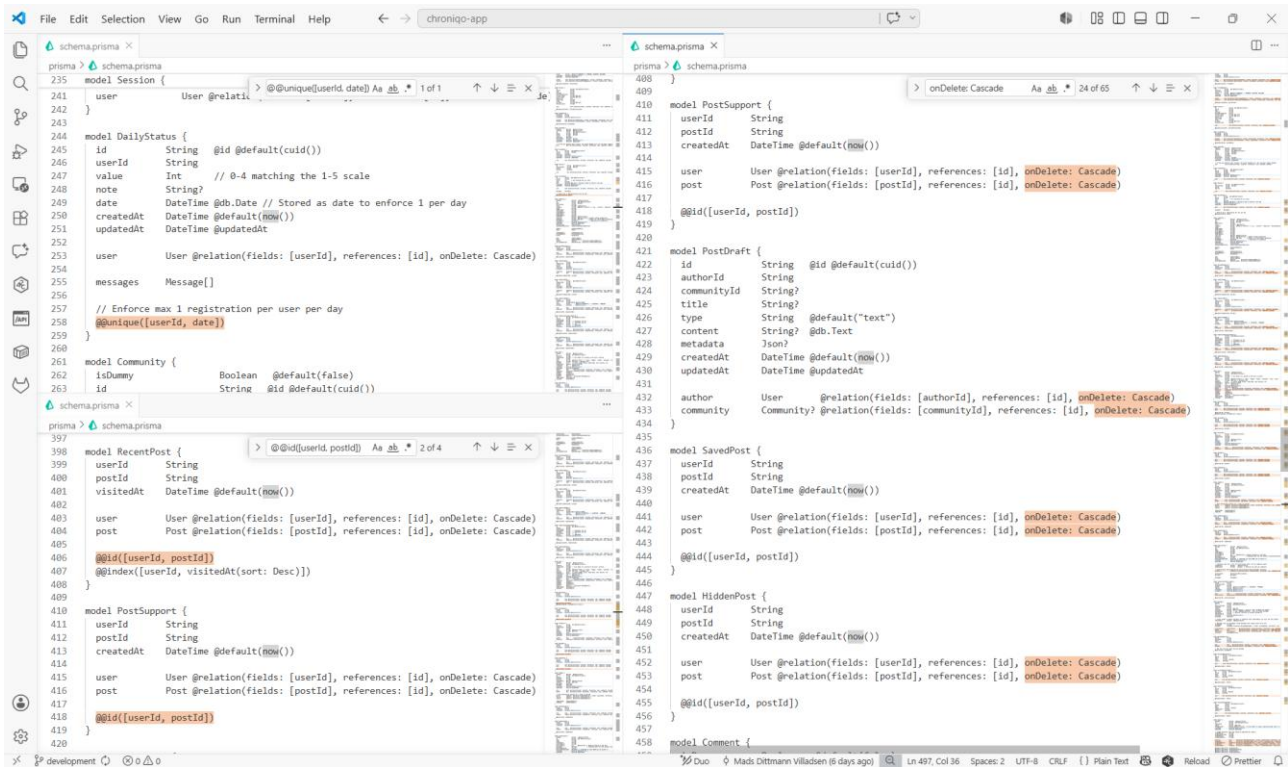
Figur 18: Feedvisningen i standardtilstand ved dagsformen Fuldt Overskud. Topnavigationen viser et notifikationsbadge med tæller ved klokke-ikonet, der signalerer afventende notifikationer. Feedet benytter infinite scroll, illustreret ved en loading-spinner i bunden af indholdsstrømmen. Samtlige interaktionsmuligheder - Støtter, Kommentarer og Del - er synlige på hvert opslag.



Figur 19: Samme feedvisning aktiveret i Low Energy Mode ved registrering af et lavt energiniveau. Notifikationsbadget i topnavigationen er skjult, og infinite scroll er erstattet af eksplicit sidepaginering ("Side 1 af X"), der begrænser den synlige informationsmængde og modvirker passivt doomscrolling.

6.2. Databasedesign og implementering

Oversættelsen fra ER-diagram til Prisma-skema er ikke uden selvstændige valg - tre implementeringsbeslutninger håndhæver centrale forretningsregler direkte i databaselaget, illustreret i Figur 20:



Figur 20: Udsnit af Chroniqos Prisma-skema, der illustrerer tre centrale implementeringsbeslutninger: den sammensatte unique constraint på `DailyStatus`, `onDelete: Cascade` på brugerrelaterede modeller og den sammensatte primærnøgle på `PostSupport`. Tilsammen håndhæver de forretningsregler om dagsformsunikalitet, GDPR-sletning og enkeltregistrering uden valideringslogik i applikationslaget.

6.2.1. Fysisk implementering i Prisma

Designprincippet bag alle tre beslutninger er det samme: forretningsregler, der ikke kan håndhæves af databasen, er regler, der kan omgås.

6.2.1.1. Dagsformens unikhedsbetingelse

Maks. én dagsformsregistrering pr. bruger pr. kalenderdag er implementeret via `@@unique([userId, date])`, hvor `date` er af typen `@db.Date` for at ignorere klokkeslættet. Service-laget er dermed tvunget til at benytte `upsert`, frem for `create` - databasen er den endegyldige garant.

6.2.1.2. GDPR via onDelete: Cascade

Retten til at blive glemt (42) er implementeret via `onDelete: Cascade` på samtlige brugerrelaterede modeller, der atomisk spreder sletningen ned igennem hele relationsgrafen ved kontosletning uden at efterlade forældreløse data¹⁷. Et bevidst undtagelsestilfælde er `GlobalBan`, der benytter `onDelete: SetNull` for at fastholde bandlysningen via e-mailadressen, selv efter kontosletning - og dermed forhindre omgåelse ved kontooprettelse.

6.2.1.3. Sammensat primærnøgle som forretningsregel

Enkeltregistreringssemantik - én bruger, én handling per entitet - er implementeret via `@@id([userId, postId])`-mønsteret på koblingstabeller¹⁸ som `PostSupport`, `MessageReaction` og `CommentSupport`. Valideringslogik på applikationsniveauet er hermed overflødig.

De komplette kodeudsnit er dokumenteret i Bilag 6, afsnit 3.1 (48).

¹⁷ Orphaned data.

¹⁸ Junction table.

6.2.2. Migrering og databasehosting

Til skemaopdateringer er `prisma db push` valgt frem for `prisma migrate dev`. Sidstnævnte genererer versionerede SQL-migrationsfiler beregnet til teamworkflows med parallelle databasegrene; for et soloprojekt med én produktionsdatabase er denne overhead et unødvendigt friktionspunkt. Det er et bevidst MVP-valg, der kan revideres ved overgang til teamudvikling.

Som managed PostgreSQL-udbyder er Neon.tech (27) valgt. Vercel Serverless Functions opretter potentielt en ny databaseforbindelse per request, et problem Neon løser via serverless connection pooling på tværs af alle instanser.

Den fulde databasekonfiguration er dokumenteret i Bilag 6, afsnit 3.2 (48).

6.3. Softwaredesign og kodestruktur

Softwarearkitekturen bag Chroniqo er struktureret efter tre designprincipper: en klar adskillelse af ansvar mellem HTTP-lag og forretningslogik, delegeret autentificering via et etableret bibliotek og proaktiv sikkerhed i kantnetværket. De følgende underafsnit beskriver realiseringen af disse principper i mappestrukturen, autentificeringsimplementeringen og rate limiting-mekanismen.

6.3.1. Mappestruktur og komponentarkitektur

Next.js-monolittens kildekode følger Separation of Concerns¹⁹-princippet med en klar opdeling mellem `src/app/api/` og `src/services/`: Route Handlers fungerer som tynde controllers, der modtager HTTP-anmodninger, validerer input via Zod og delegerer til service-laget. Al forretningslogik er samlet i `src/services/` og er fuldstændig uafhængig af HTTP-transportsystemet - en arkitektur, der direkte muliggør enhedstest uden simulering af HTTP-kontekst, og som er forudsætningen for de 97 Jest-testsuites (jf. afsnit 7.2).

Alle beskyttede frontend-ruter er samlet under én `(protected)`-rutegruppe, der håndhæver autentificeringskravet strukturelt ét sted frem for som gentagne session-checks i hver komponent. De tre applikationsdækkende React Contexts - `DailyStatusContext`, `ThemeContext` og `SessionProvider` - er wrappers i rodlayoutet og tilgængelige for hele komponenttræet.

Den fulde annoterede mappestruktur fremgår af Bilag 6, afsnit 4.1 (48).

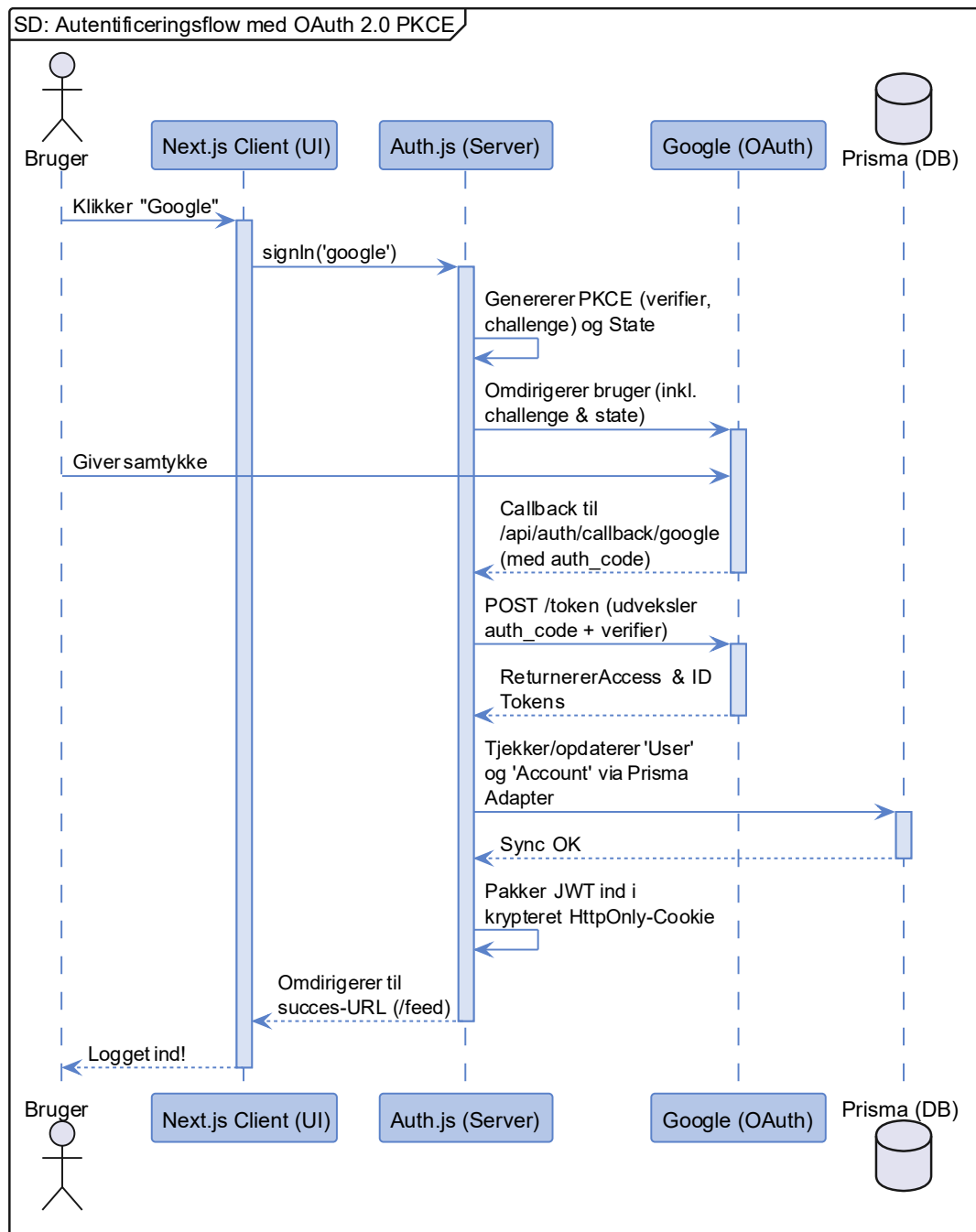
¹⁹ Separation of Concerns (SoC).

6.3.2. Implementering af autentificering

Autentificering er implementeret via Auth.js (NextAuth v5) med to parallelle strategier: credentials-flow via `CredentialsProvider` med e-mail og adgangskode, og Google OAuth 2.0 PKCE-flow, hvis forløb er illustreret i Figur 21. JWT-strategien er et arkitekturkrav ved brug af `CredentialsProvider` - tokens pakkes ind i krypterede `HttpOnly`-cookies, immune over for XSS og CSRF²⁰.

En bevidst konfigurationsbeslutning er `allowDangerousEmailAccountLinking: true`, der giver eksisterende e-mailbrugere mulighed for at koble en Google-konto til deres eksisterende konto; trade-off'et er velkendt (brugeroplevelse mod teoretisk account takeover-risiko) og vurderes acceptabelt i MVP-konteksten.

²⁰ CSRF (Cross-Site Request Forgery).



Figur 21: Sekvensdiagram over Auth.js' OAuth 2.0 PKCE-flow med Google som identitetsudbyder. Flowet illustrerer, hvordan Next.js-serveren genererer et PKCE-par, omdirigerer brugeren til Googles samtykkeskærm, og efter callback udveksler `authorization code` og `code verifier` for Access- og ID-tokens. Sessionen afsluttes med udstedelsen af en krypteret `HttpOnly-cookie`, der indeholder JWT'en.

`signIn`-callbacket tjekker `GlobalBan`-tabellen ved hvert loginforsøg og afviser bandlyste brugere med et token til kontosletning, der giver dem mulighed for at fjerne egne data uden aktiv session. `jwt`-callbacket propagerer brugerdefinerede felter (`username`, `onboarded`, `role`, `avatarEmoji`) til JWT'en og understøtter direkte klientopdatering via `trigger === "update"` - eksempelvis til at sætte `onboarded: true` umiddelbart efter onboarding-flowet uden at brugeren skal logge ind på ny.

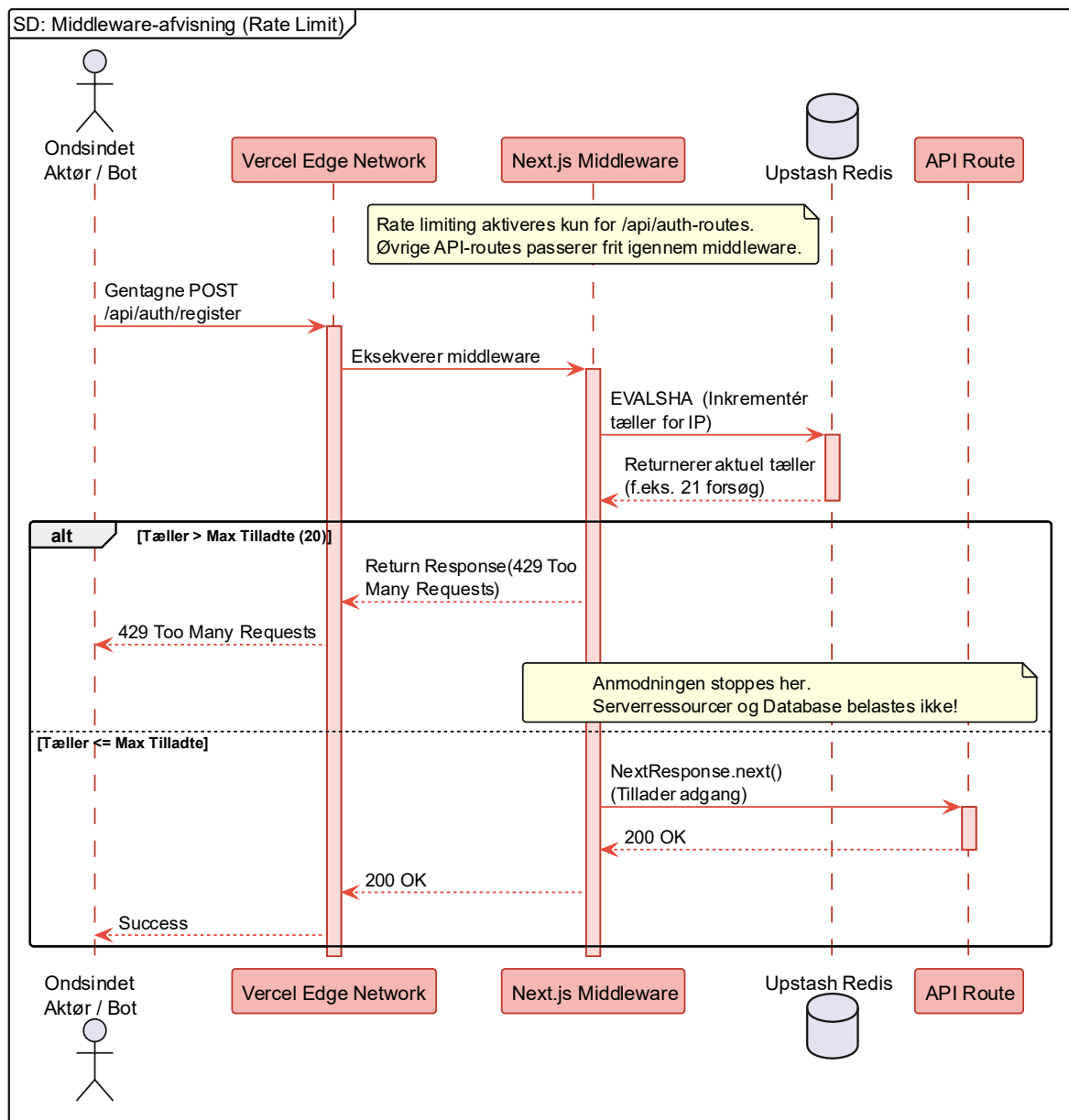
Den fulde `Auth.js`-konfiguration er dokumenteret i Bilag 6, afsnit 4.2 (48).

6.3.3. Håndtering af rate limiting og JWT-revokation

`src/proxy.ts` er systemets Next.js Edge Middleware og varetager to ansvarsområder: rate limiting på autentificeringsendpoints og i18n-lokaliseringsrouting.

Rate limiting er afgrænset til `/api/auth`-ruter efter fail fast-princippet (56) - disse er de primære mål for brute-force-angreb og credential stuffing²¹, mens øvrige endpoints er beskyttet bag autentificeringslaget. Algoritmen er et sliding window med 20 anmodninger per 10 sekunder per IP (valgt frem for fast window for at undgå burst-spikes ved vinduesgrænsen) (57); angrebsscenariet, hvor grænsen overskrides og en HTTP 429 returneres, er modelleret i Figur 22. En separat limiter på 20 anmodninger per time beskytter Niqo/Gemini-endpointene mod omkostningseskalering. Er Redis utilgængeligt, degraderer systemet gracefully.

²¹ Credential stuffing.



Figur 22: Sekvensdiagram over rate limiting-flowet i Chroniqo. Edge Middleware i Vercels kantnetværk forespørger Upstash Serverless Redis om den aktuelle anmodningstæller for den pågældende IP-adresse - afgrænset til `/api/auth-routes` som det primære mål for brute-force og credential stuffing. Overskrides grænsen på 20 anmodninger per 10 sekunder, returneres HTTP 429 øjeblikkeligt. Er Redis utilgængeligt, degraderer systemet gracefully og tillader anmodningen at passere. Diagrammet anvender et rødt farvetema for at understrege, at der her modelleres et angrebsscenario forsat af en ondsindet aktør/bot.

JWT-tokens er stateless og kan ikke tilbagekaldes (58). Løsningen er en Redis-baseret ban-flagmekanisme: ved bandlysning skrives `banned:{userId}` til Redis med Time To Live (TTL) svarende til JWT'ens levetid, og BanWatcher-komponenten poller serveren hvert 30. sekund for øjeblikkelig afmelding - en pragmatisk mellemløsning, der begrænser vinduesperioden til maksimalt 30 sekunder uden databaseopslag ved hvert request.

Den fulde implementering er dokumenteret i Bilag 6, afsnit 4.3 (48).

6.4. Automatiseret deployment

Det overordnede princip bag deploymentstrategien er **NoOps**: ved udelukkende at benytte managed cloud-tjenester - Vercel (22) til applikationshosting, Neon.tech (27) til PostgreSQL (26) og Upstash (29) til Redis (28) - elimineres al manuel serverdrift. Deployment til produktion sker automatisk ved hvert merge til `production`-branchen: når alle kvalitetsgates er succesfuldt gennemført - sikkerhedsaudit, linting, typetjek og testsuiten - eksekveres `vercel deploy --prebuilt --prod`, og ved samme merge bygges og pushes et Docker-image tagget med commit-SHA til Docker Hub (35) for præcis versionsgendannelse.

Den fulde CI/CD-pipeline, der resulterer i applikationen tilgængelig på <https://chroniqo-app.vercel.app>, er dokumenteret i Bilag 6, afsnit 5.1 (48); loginoplysninger til en delt demokonto med administratorrettigheder fremgår af Bilag 8: UI-Dokumentation (55).

6.5. Delkonklusion for design og implementering

Med implementeringen på plads er de arkitektoniske intentioner fra afsnit 5 omsat til konkrete beslutninger på tværs af fire lag.

På brugergrænsefladen er kognitiv skånsomhed realiseret via et konsistent designsystem med dæmpede dagsformsfarver, dual-theme og adaptiv UI-logik centraliseret i `DailyStatusContext`. *Low Energy Mode* reducerer automatisk den kognitive belastning ved lave energiniveauer uden aktiv handling fra brugeren.

I databaselaget er tre centrale forretningsregler håndhævet i Prisma-skemaet: unikhedsbetingelsen på `DailyStatus`, GDPR-sletning via `onDelete: Cascade` og enkeltregistrering via sammensatte primærnøgler. Valget af `prisma db push` er et bevidst MVP-valg, der reducerer overhead for enkeltmandsprojektet.

Softwarearkitekturen følger Separation of Concerns med tynde Route Handler-controllers og et isoleret service-lag, der muliggør enhedstest af forretningslogik. Autentificeringen kombinerer JWT-strategi og PKCE med et `signIn`-callback, der løser ban-tjek og OAuth-verifikation i én operation, mens Redis-ban-mekanismen adresserer JWT-revokationsproblemet hensigtsmæssigt. Deployment er fuldt automatiseret via en NoOps-strategi og danner direkte grundlag for testdækningen i afsnit 7.

7. Test og validering

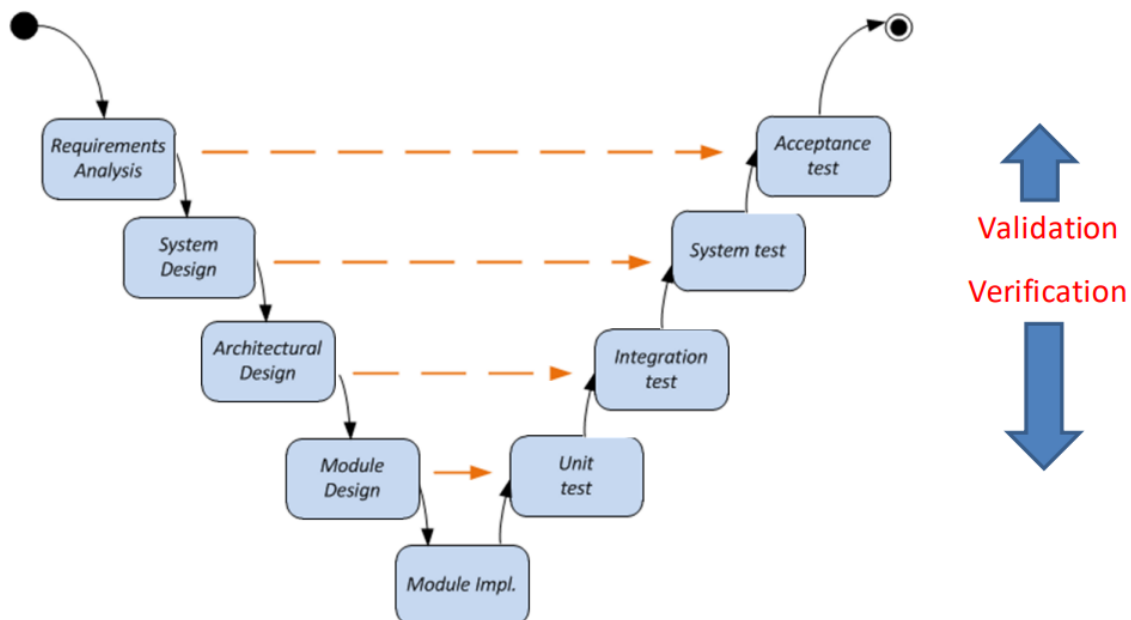
Et systems kvalitet kan kun dokumenteres, hvis kravene, der motiverede det, er direkte sporbare til de tests, der godkender det. Testindsatsen for Chroniqo er struktureret efter V-modellens fire niveauer med sporbarhed fra User Stories i afsnit 2 (kravspecifikation) til godkendte accepttestresultater - og med en brugertest-protokol, der stiller det ene spørgsmål, automatiserede tests ikke kan besvare: om *Low Energy Mode* faktisk opleves som kognitiv skånsom i praksis.

Den fulde metodebeskrivelse og detaljerede testresultater fremgår af Bilag 7: Test & Validering (59).

7.1. Teststrategi (V-modellen)

Teststrategien, illustreret i Figur 23, er baseret på V-modellen (60), der adskiller de to centrale kvalitetsspørgsmål: *er systemet bygget teknisk korrekt?* og *løser systemet brugerens reelle behov?* Som enkeltmandsprojekt er testindsatsen prioriteret strategisk med fokus på at sikre de mest kritiske dele af systemet med de tilgængelige ressourcer.

V-Model and Test levels



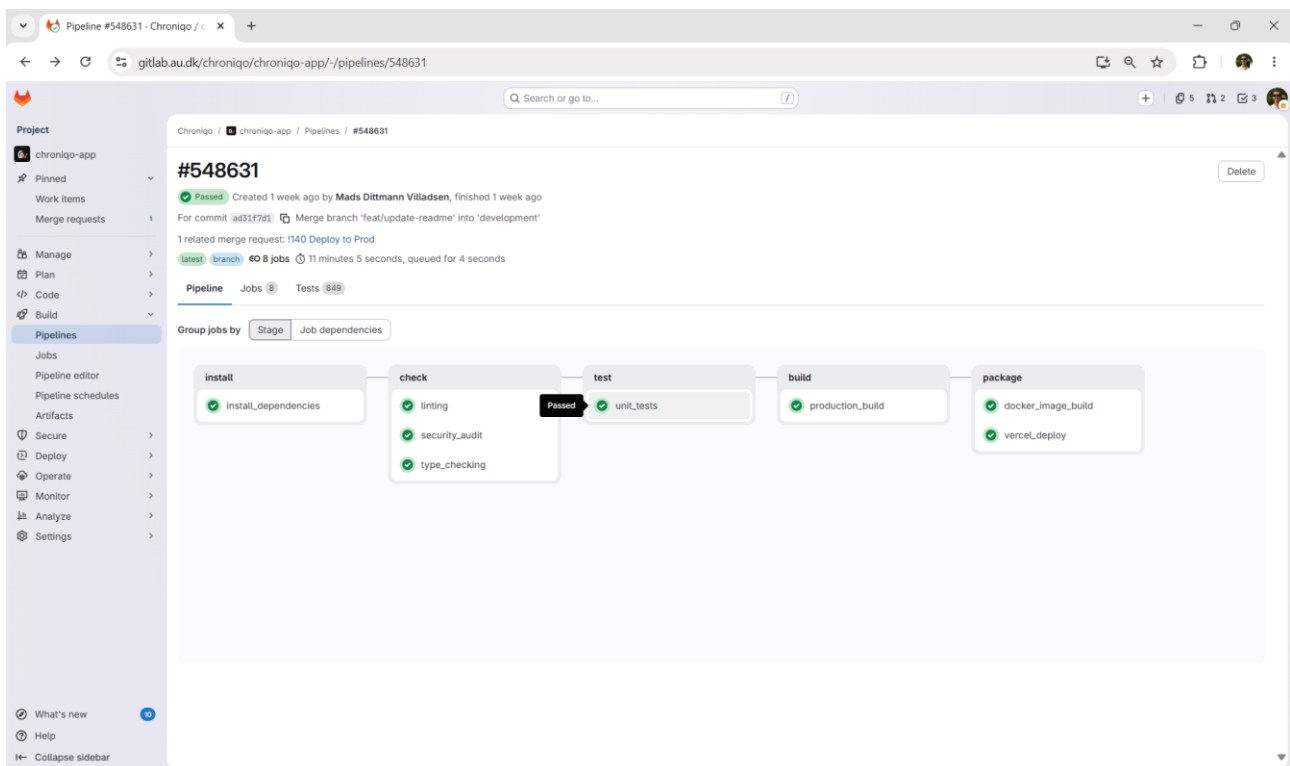
Figur 23: V-modellens hierarki med to sider: Den venstre, nedadgående side repræsenterer Chroniqos specifikations- og udviklingsfaser - fra Requirements Analysis (kravspecifikation med User Stories, MoSCoW og Gherkin-scenarier, afsnit 2) over System Design og Architectural Design (systemarkitektur og teknologistak, afsnit 5) til Module Design og Module Implementation (service-lag og konkret implementering, afsnit 6) i bunden af V'et. Den højre, opadgående side repræsenterer de korresponderende testniveauer: unit- og integrationstest via Jest (modul/unittest-niveau), databasetest via Prisma Studio (integrationstestniveau), manuel accepttest af alle 68 User Stories (systemtestniveau), og Think-Aloud-brugertesten (accepttestniveau øverst). De to pile til højre afspejler skellet mellem verifikation (er systemet bygget korrekt?) på de lavere niveauer og validering (løser det brugerens behov?) på det øverste niveau. Kilde: Tilpasset fra Systemtest - Introduction to Systems Engineering I2ISE (60).

Testindsatsen er fordelt over fire niveauer: automatiserede unit- og integrationstest via Jest (36) verificerer forretningslogik og API-autorisation; databasetest via Prisma Studio (25) bekræfter kritiske constraints i produktionsmiljøet; manuel accepttest via brugergrænsefladen på Vercel verificerer alle 68 User Stories; og en kvalitativ Think-Aloud-brugertest (61), der validerer den oplevede brugeroplevelse, hvis protokol er fuldt specificeret, men ikke eksekveret inden for tidsrammen (jf. afsnit 7.6).

Den fulde teststrategi med metodebeskrivelse fremgår af Bilag 7 (59).

7.2. Automatiserede tests (Unit og Integration)

Systemets forretningslogik er testet med Jest (36), integreret direkte i GitLab (16) CI/CD-pipelinen og eksekveret automatisk ved hvert commit. Testsuiten udgøres af 858 tests fordelt over 97 testfiler organiseret i fire kategorier: API-tests (78 suites) verificerer Route Handlers' autorisations- og valideringslogik; service-tests (13 suites) verificerer forretningslogikken i isolation via `jest-mock-extended` med 100 % typesikker mocking af Prisma-klienten; utility-tests (5 suites) dækker deterministiske hjælpefunktioner, herunder PII-filteret og søgekortlæggerne; middleware-tests (1 suite) dækker Edge Middleware-lagets lokaliserings- og bandlysningskontrol. Alle tests er struktureret efter AAA-mønsteret (Arrange-Act-Assert) (62); Figur 24 dokumenterer den succesfulde CI-kørsel med samtlige 858 tests bestået:



Figur 24: Skærbillede af en succesfuld GitLab CI/CD-pipeline-kørsel for Chroniqo, hvor samtlige otte jobs er markeret som "Passed". Det dedikerede `unit_tests`-job i test-fasen er fremhævet for at dokumentere, at testsuiten gennemføres succesfuldt som en obligatorisk betingelse for efterfølgende build og deployment til Vercel. JUnit-rapporten, der genereres via Jest's `--ci`-flag, fremgår som artifact tilknyttet jobbet.

Samtlige 858 tests gennemføres fejlfrit. Den opnåede kodedækning fremgår af Tabel 6:

Tabel 6: Samlet kodedækningsgrad opnået ved kørsel af testsuiten med `jest --coverage`. Tabellen viser dækningsgraden opdelt på de fire målte dimensioner samt antallet af målepunkter, der er dækket i forhold til den samlede kodebase.

Samlet kodedækningsgrad opnået ved kørsel af testsuiten		
Dækningsmål	Dækning	Dækkede / Total
Statements	83,44 %	14.186 / 17.001
Branches	77,25 %	1.770 / 2.291
Functions	83,69 %	267 / 319
Lines	83,44 %	14.186 / 17.001

Den resterende 16,6 % udgøres primært af defensive fejlhåndteringsgrene i Route Handlers - eksempelvis `catch`-blokke, der returnerer `500 Internal Server Error` ved databasefejl - der kræver simulering af infrastrukturfejl for at aktiveres under test; branches-dækningen på 77,3 % afspejler samme mønster i `null`-checks og asynkrone fejlstier - eksempelvis grene, der håndterer `null`-returværdier fra `findOne`-kald, der ikke eksisterer i databasen under normale testbetingelser.

Den fulde oversigt over opsætning, metode, terminaloutput m.m. fremgår af Bilag 7, afsnit 3 (59).

7.3. Databasetest og dataintegritet

Ud over de automatiserede tests er fire kritiske database-constraints verificeret manuelt via Prisma Studio (25) direkte mod produktionsdatabasen på Neon.tech (27) for at bekræfte korrekt adfærd under reelle betingelser - ikke blot i isolation med en mocket Prisma-client.

De fire verificerede constraints er:

1. `@@unique([userId, date])` på `DailyStatus` sikrer maks. én dagsformsregistrering pr. bruger pr. kalenderdag via `upsert`-semantik - **OK**
2. `onDelete: Cascade` på samtlige brugerrelaterede relationer bekræfter GDPR's ret til at blive glemt: atomisk sletning af al tilknyttet brugerdata over otte tabeller, suppleret med Redis-sessionsinvalidering via Upstash - **OK**
3. `onDelete: SetNull` på `GlobalBan.userId` bekræfter, at et aktivt platformban fastholdes via e-mailadressen, selv efter den tilknyttede brugerkonto er slettet - **OK**
4. Cron-jobbet²² `GET /api/cron/prune-chats` identificerer og sletter korrekt udløbne samtaler via `expiresAt-OR`-betingelse med kaskadesletning af tilknyttede rækker - **OK**

Databasetesten bekræfter, at systemet kan eksekvere GDPR-kravene på databaseniveau - ikke blot som applikationslogik.

Den fulde verifikation med trin-for-trin dokumentation og skærmbilleder fra Prisma Studio fremgår af Bilag 7, afsnit 4 (59).

²² Cron-job.

7.4. Accepttest - manuel systemtest

Accepttesten verificerer, at alle specificerede User Stories er opfyldt set fra slutbrugerens perspektiv. Testene er udført manuelt via brugergrænsefladen i produktionsmiljøet på Vercel for end-to-end-dækning af hele teknologistakken. I alt er 68 User Stories over de fire Epics testet; den samlede teststatus fremgår af Tabel 7:

***Tabel 7:** Samlet overblik over acceptteststatus for alle 68 User Stories fordelt på de fire Epics. Kolonnen "Bestået" angiver User Stories, der er godkendt på samtlige testtrin ved testforløbets afslutning. Alle User Stories er testet mod de tilhørende Gherkin-scenarier specificeret i kravspecifikationens accepttestspecifikation (jf. afsnit 2.5).*

Samlet overblik over acceptteststatus for alle 68 User Stories				
Epic	User Stories testet	Bestået	Ikke bestået	MoSCoW-dækning
Epic 1: Autentificering & Profil	19 (US1.1-US1.19)	19	0	Alle M, S og C
Epic 2: Dagsform & Adaptivt UI	16 (US2.1-US2.16)	16	0	Alle M, S og C
Epic 3: Socialt Fællesskab	19 (US3.1-US3.19)	19	0	Alle M, S og C
Epic 4: Platform Moderation	14 (US4.1-US4.14)	14	0	Alle M, S og C
Total	68	68	0	

Resultaterne afspejler et iterativt testforløb, ikke et fejlfrit første gennemløb: fejl og edge cases opstod løbende og krævede koderettelser, inden en User Story kunne godkendes. To centrale eksempler illustrerer testforløbets dybde:

7.4.1. Dagsformsregistrering (US2.1 - Must)

Verificerer, at en bruger kan registrere sin dagsform ved første login på en ny kalenderdag, og at systemet skjuler `DailyStatusInterceptor`'en for det resterende 24-timers vindue. Testen bekræftede korrekt håndhævelse af `@@unique([userId, date])` og korrekt `sessionStorage`-håndtering på klienten. **Verificeret og godkendt.**

7.4.2. Anonym interaktion (US3.10 - Must)

Verificerer, at en bruger kan oprette et opslag anonymt i et fællesskab, og at moderatorer og administratorer fortsat kan identificere den reelle afsender via `CommunityAnonymousIdentity`-tabellen, mens øvrige brugere udelukkende ser den anonyme identitet. **Verificeret og godkendt.**

Samtlige Must-have-krav er verificeret og godkendt, hvilket bekræfter, at MVP-scopet er fuldt realiseret. Den fulde testprotokol med trin-for-trin gennemgang af alle 68 User Stories og skærmbilledokumentation fremgår af Bilag 7, afsnit 5 (59).

7.5. Test af ikke-funktionelle krav (FURPS+)

De ikke-funktionelle krav specificeret i FURPS+-modellen (jf. afsnit 2.4.1) er verificeret parallelt med accepttestene. Samlet teststatus per krav fremgår af Tabel 8:

Tabel 8: Samlet teststatus for samtlige ikke-funktionelle krav (FURPS+) organiseret efter kvalitetskategori. Tabellen angiver krav-ID, kort kravbeskrivelse, MoSCoW-prioritering og verificeret teststatus ved MVP-leverance. Krav markeret "Delvist opfyldt" er uddybet i den efterfølgende brødtekst.

Samlet teststatus for samtlige ikke-funktionelle krav (FURPS+)				
Kategori	Krav-ID	Krav (kort beskrivelse)	MoSCoW	Status
Usability				
	UR1	Navigation til kernefunktioner i maks. 2-3 klik	M	OK
	UR2	Responsivt layout ved ≥ 500 px skærbredde	M	OK
	UR3	WCAG AA-kontraststandarder	S	Delvist opfyldt
	UR4	Øjeblikkelig og tydelig visuel feedback	M	OK
Reliability				
	RR1	Datapersistering med 100 % integritet	M	OK
	RR2	Graceful degradation ved tab af forbindelse	S	OK
	RR3	Konsistent datatilstand ved samtidige handlinger	S	Arkitektonisk opfyldt
Performance				
	PR1	Feed-indlæsning under 2 sekunder på 4G	S	Delvist opfyldt

	PR2	50 samtidige brugere uden mærkbar forringelse	C	Ikke testet (arkitektonisk begrundet)
	PR3	AI-støttebot (Niqo) svartid under 5 sekunder	C	OK
Supportability				
	SR1	Modulær kodebase (Open-Closed Principle)	M	OK
	SR2	README muliggør onboarding under 60 min	S	OK
	SR3	Struktureret logging af systemhændelser	S	OK

7.5.1. Usability (U)

Alle tre Must-krav er fuldt opfyldt. UR3 (WCAG AA) er delvist opfyldt: brandfarven (#E65C69) opnår et kontrastforhold på 3,2:1, som overholder AA for UI-komponenter og store elementer (50), men ikke for brødtekst - brandfarven bruges udelukkende på CTA-knapper og aldrig til brødtekst, hvorfor WCAG AA er overholdt i samtlige faktiske anvendelseskontekster. Et Lighthouse-audit opnår 85/100.

7.5.2. Reliability (R)

RR1 er direkte verificeret via databasetesten i afsnit 7.3. RR2 er testet via Chrome DevTools' off-linetilstand; optimistisk UI rulles korrekt tilbage, og formularindhold bevares. RR3 er verificeret arkitektonisk: PostgreSQL's ACID²³-transaktionsgarantier og Prismas implicitte transaktionshåndtering sikrer konsistent tilstand ved samtidige skrivinger.

7.5.3. Performance (P)

PR1 er delvist opfyldt: Lighthouse Simulated Throttling viser LCP²⁴ på 1,4 sek., mens Chrome DevTools Applied Throttling (Fast 4G) måler 3,46 sek. som følge af en client-side SWR-vandfaldsstruktur, der sekventielt afvikler HTML-levering, React-hydrering, SWR-datafetching og billedrendering, inden LCP opnås. PR2 er et Could-krav og ikke load-testet; arkitektonisk adresseret via Vercels horisontale skalering og Upstash Redis. PR3 (Niqo) er målt til 3,46 sek. i produktionsmiljøet.

7.5.4. Supportability (S)

SR1 er verificeret via Next.js App Router-strukturens adskillelse af UI, services og API; minigame-modulet er tilføjet som selvstændigt modul uden ændringer i eksisterende servicelag. SR2 og SR3 er begge verificeret.

Den fulde FURPS+-testprotokol med skærbilledokumentation fremgår af Bilag 7, afsnit 6 (59).

²³ ACID (Atomicity, Consistency, Isolation, Durability).

²⁴ LCP (Largest Contentful Paint).

7.6. Brugertest (validering)

Brugertesten udgør det øverste valideringsniveau i V-modellen og skal afdække, om *Low Energy Mode* og den samlede brugeroplevelse opleves som kognitiv skånsom af repræsentative slutbrugere - en validering, der ikke kan løftes af tekniske tests alene.

Protokollen er designet som en kvalitativ Think-Aloud-test²⁵ (61) med fem deltagere: tre rekrutteret fra relevante patientforeninger og to med varierende it-erfaring, i tråd med Nielsens estimat om, at fem deltagere afdækker ca. 85 % af alle usability-problemer (61, 63). Tre kernescenarier dækker henholdsvis onboarding-flowet, dagsformsregistrering med observation af *Low Energy Mode* og anonym fællesskabsinteraktion via Rich Text Editor (platformens tekstformateringsværktøj til opslag og kommentarer). Succeskriteriet for det centrale scenarie er, at minimum 4 ud af 5 deltagere spontant bemærker den visuelle ændring ved *Low Energy Mode* uden vejledning (tilknyttet krav US2.13 og UR4).

Som følge af de helbredsmæssige udfordringer beskrevet i afsnit 4.2.1 var det ikke muligt at eksekvere testen inden for den planlagte tidsramme. Protokollen er dog fuldt specificeret og klar til umiddelbar eksekvering, og udgør et konkret emne for fremtidigt arbejde (jf. afsnit 10.1).

Den fulde testprotokol med deltagerprofil, observationspunkter og succeskriterier fremgår af Bilag 7, afsnit 7 (59).

²⁵ Think-Aloud-test.

7.7. Delkonklusion for test og validering

Testindsatsen for Chroniqo hviler på fire komplementære niveauer, der tilsammen dokumenterer systemets kvalitet fra isolerede enheder til samlet brugeroplevelse.

På verifikationsniveauet bekræfter 858 automatiserede tests med 83,4 % statement coverage, at forretningslogikken er robust og fri for regressioner. Databasetesten dokumenterer, at de fire kritiske constraints fungerer korrekt i produktionsmiljøet, herunder GDPR-kompliant kontosletning og chatudløb. Den manuelle systemtest bekræfter på end-to-end-niveau, at alle 68 User Stories er implementeret og fungerer som specificeret. Samtlige Must-have FURPS+-krav er opfyldt; to krav er delvist opfyldt (UR3 og PR1) med veldokumenterede arkitektoniske forklaringer, og det eneste krav der ikke er formelt testet (PR2) er et Could-krav med arkitektonisk begrundelse.

På valideringsniveauet er brugertestprotokollen fuldt specificeret men afventer eksekvering.

Samlet set er Chroniqos MVP verificeret og klar til release. Systemet opfylder fuldt ud Must-have-omfanget, og de identificerede begrænsninger er karakteriseret ved tekniske afvejninger frem for fejl i implementeringen.

8. Diskussion

Chroniqo er nu bygget, testet og deployeret. Hvad beviser et system, der er verificeret på alle tekniske niveauer - men endnu ikke har mødt sin bruger? Formålet er ikke at gentage resultater, men at fortolke dem: at evaluere, i hvilken grad det realiserede system indfrier de opstillede krav, at reflektere kritisk over de arkitektoniske og metodiske valg, der viste sig at holde - og dem, der i bagklogskabens lys ville have fortjent et andet svar - og afslutningsvis at give en åben og ærlig redegørelse for de menneskelige rammebetingelser, projektet er gennemført under.

8.1. Kravopfyldelse og MVP-realisering

Projektets primære succeskriterium har fra begyndelsen været klart defineret: at levere en fuldt funktionel MVP, der opfylder samtlige Must-have-krav. Det resultat er nået - og endda oversteget. Samtlige 34 Must-, 23 Should- og 11 Could-krav er verificeret og godkendt gennem den samlede testindsats beskrevet i afsnit 7.

Tabel 9 viser et repræsentativt udsnit af User Stories fordelt på alle fire Epics og alle tre MoSCoW-kategorier. Den fulde verifikation af samtlige 68 User Stories fremgår af Bilag 7, afsnit 5 (59).

Tabel 9: Repræsentativt udsnit af verificerede User Stories på tværs af Chroniqos fire Epics og tre MoSCoW-kategorier. Samtlige 68 User Stories er godkendt ved testforløbets afslutning; tabellen er et udsnit og ikke en udtømmende liste.

Repræsentativt udsnit af verificerede User Stories på tværs af Chroniqos fire Epics og tre MoSCoW-kategorier				
Krav-ID	Krav (kort beskrivelse)	Epic	MoSCoW	Status
US1.1	Kontooprettelse med e-mail og adgangskode	1	Must	Opfyldt
US1.3	Google OAuth 2.0 login	1	Should	Opfyldt
US1.9	GDPR-sletning (retten til at blive glemt)	1	Must	Opfyldt
US2.1	Dagsformsregistrering (skala 1-5, én gang pr. dag)	2	Must	Opfyldt
US2.13	Adaptivt brugerinterface (<i>Low Energy Mode</i>)	2	Must	Opfyldt
US3.1	Direkte beskeder og tidsbegrænsede gruppesamtaler	3	Must	Opfyldt
US3.10	Anonym interaktion i fællesskaber	3	Must	Opfyldt
US3.18	Niqo AI-støttebot med PII-filtrering	3	Could	Opfyldt
US4.1	Global bandlysning og mute-funktion	4	Must	Opfyldt
US4.13	Minispil integreret i beskedsamtaler	4	Could	Opfyldt

8.1.1. FURPS+-krav: konstruktiv evaluering af de delvist opfyldte krav

To ikke-funktionelle krav er i afsnit 7.5 vurderet som delvist opfyldt, og begge fortjener en kort diskussion her.

8.1.1.1. UR3 (WCAG-kontraststandarder - Should)

Lighthouse-audits viser, at dagsformsfarvernes dæmpede pastelnuancer på visse tekst-kombinationer ikke opnår WCAG 2.1 AA's 4,5:1-kontrastkrav. Designvalget er bevidst: intuitiv, differentieret energisignalering er prioriteret over fuld compliance i MVP-fasen. Det er et valg, jeg ville have screenet systematisk tidligere i designprocessen - et kontrasttjek i et tidligt Figma-prototypestadie ville have åbnet for finere justeringer, inden tokens var låst i globals.css. Vejen frem er dokumenteret i afsnit 10.4.

8.1.1.2. PR1 (Feedindlæsning under 2 sekunder på 4G - Should)

Forskellen mellem Simulated Throttling (LCP 1,4 sek.) og Applied 4G-throttling (LCP 3,46 sek.) er ikke en fejl, men afspejler SWRs client-side vandfaldsstruktur: HTML leveres, React hydreres, og SWR henter feeddata asynkront, inden LCP måles. Det er et pragmatisk MVP-valg med en veldefineret teknisk opgraderingsvej beskrevet i afsnit 10.6.

8.1.2. Den ikke-eksekverede brugertest

Den eneste åbne validering i projektets testgrundlag er brugertesten beskrevet i afsnit 7.6. Testprotokollen er fuldt specificeret - deltagerprofil, scenarier og kvantificerbare succeskriterier er på plads - men eksekveringen var ikke mulig inden for projektets tidsramme som følge af de helbredsmæssige udfordringer, der uddybes i afsnit 8.4. Det er vigtigt at understrege, hvad dette betyder: den tekniske verifikation af *Low Energy Mode* er solid på tre niveauer (automatiserede tests, databasetest og manuel accepttest), men den empiriske bekræftelse af, at det adaptive interface reelt opleves som kognitivt skånende af brugere fra målgruppen, mangler. Det er den ene afgørende åbne hypotese. Protokollen er klar til umiddelbar eksekvering og udgør det mest presserende punkt i fremtidigt arbejde.

8.2. Tekniske kompromiser og arkitektoniske refleksioner

Et projektforsløb producerer ikke blot en kodebase - det producerer erfaring. De tekniske beslutninger, der viste sig at holde, dem der ville have fortjent et andet svar, og én feature der bød på mere designmæssig dybde end forventet, fortjener en nærmere refleksion.

8.2.1. Architectural Simplification: beslutningen der holdt

Valget om at samle frontend og backend i én Next.js-monolit (jf. afsnit 4.3.2) var den mest gennemgribende arkitektoniske beslutning i projektet - og den viste sig at være rigtig. End-to-end typesikkerhed via Prisma betød, at domæneobjekter som `DailyStatus`, `Post` og `CommunityAnonymousIdentity` var umiddelbart tilgængelige på tværs af hele systemet uden manuelle synkroniseringslag. En enkelt CI/CD-pipeline dækkede hele stakken. CORS-problematikker eksisterede simpelthen ikke. I bakspejlet er det klart, at en microservice-tilgang - deployed i et Kubernetes-cluster, som jeg kender fra tidligere valgfagskurser - under de givne rammebetingelser ville have forsinket projektet væsentligt og sandsynligvis ført til et smallere funktionelt omfang.

8.2.2. Short-polling til chat: et bevidst kompromis der holdt sit løfte

Short-polling via SWR (jf. afsnit 4.3.2 og Bilag 4, afsnit 2.6 (20)) var et bevidst arkitektonisk trade-off, motiveret af serverless-infrastrukturens manglende understøttelse af persistente TCP-forbindelser. Løsningen virkede fuldt ud til de formål, der er verificeret inden for MVP'ens scope - samtaler fungerede, latensen var acceptabel, og polling-mekanismen var transparent for slutbrugeren. PR2-kravet om 50 samtidige brugere er arkitektonisk adresseret via Vercel Serverless Functions' automatiske horisontale skalering og Upstash Redis, men er ikke formelt load-testet (jf. afsnit 7.5.3). Skaleringsbarrieren er dog reel: databasebelastningen vokser lineært med antallet af aktive samtaler og polling-frekvensen (kortere interval - flere forespørgsler), og latensen er arkitektonisk uundgåelig inden for det nuværende setup - den konkrete opgraderingsvej er beskrevet i afsnit 10.2.

8.2.3. Kommentar-nesting begrænset til ét indlejrningsniveau

Beslutningen om at begrænse kommentarsporet til ét indlejrningsniveau - et direkte svar kan besvares, men det svar kan ikke selv besvares - var motiveret af to forhold: databasens ON DELETE CASCADE-logik, der ved dyb rekursion kan producere uforudsigelig adfærd i form af kædede sletninger, der låser tabellen eller overskrider databasens rekursionsgrænse, og den kognitive belastning, som dybe trådstrukturer pålægger brugere med begrænset energioverskud. Begrænsningen er implementeret som en selvrefererende relation i Comment-modellen med et enkelt parentId-felt. I bagklogskabets lys vurderer jeg, at det var det rigtige valg for MVP'en - men grænsen bør evalueres kvalitativt i brugertesten, da det er muligt, at to niveauer ville have givet en bedre diskussionsdynamik uden at øge kompleksiteten markant.

8.2.4. Minispil: en uventet designmæssig udfordring

Minispillene - Tic-Tac-Toe, Connect Four og Knucklebones - er et Could-krav og repræsenterer på mange måder projektets mest atypiske feature. Rationalet bag dem er konkret: korte, lavintensive interaktioner i beskedudvekslinger er et let tilgængeligt socialt element for brugere med begrænset energioverskud, og de adskiller Chroniqo fra platforme, der udelukkende fokuserer på tekstbaseret støtte og deling.

Den tekniske udfordring viste sig at ligge i modelleringen af hvert spils interne tilstandslogik. Hvert spil har én Minigame-entitet i databasen med et JSON-tilstandsfelt, der lagrer spillets fulde board-repræsentation - strukturen er spilspecifik, men protokollen er ensartet: en udfordring starter i PENDING, accepteres til ACTIVE og afsluttes i enten COMPLETED, CANCELLED eller DECLINED. LIVE-tilstanden, der kræver hurtigere polling end den normale chat (3 sek. vs. 5 sek.), nødvendiggjorde en dedikeret live-game-watcher-komponent og en separat live-invitations-Route Handler.

Lydintegration - separate lydeffekter og baggrundsmusik per spil hentet fra åbne kilder - tilføjede et ekstra præsentationslag, der krævede nøje styring af Web Audio API-tilstand for at undgå overlap. Det var en givende og anderledes teknisk udfordring, og den type feature illustrerer, at Chroniqo er designet til at være mere end bare et støtteforum.

8.2.5. PII-filteret i Niqo: pragmatisk, men ikke ufejlbarligt

Niqo-integrationens privacy-by-design-tilgang - regex-baseret filtrering af navne, e-mailadresser, telefonnumre og CPR-numre server-side inden afsendelse til Gemini API - er et solidt fundament, men det har en iboende begrænsning: regex matcher mønstre, ikke semantik. En bruger, der skriver sit CPR-nummer med skråstreg i stedet for bindestreg, eller et telefonnummer med mellemrum, kan ved en fejl passere filteret. En LLM-baseret PII-klassifikation som alternativ til regex ville give højere recall (andelen af PII, der faktisk identificeres), men introducerer latens, kompleksitet og yderligere API-afhængigheder, der er vanskeligt forsvarlige i en MVP. Regex-tilgangen er det rigtige valg til dette stadie, men den bør suppleres med yderligere mønstre og eventuelt erstattes af en dedikeret NER-model²⁶ i en moden version af platformen.

²⁶ NER (Named Entity Recognition).

8.3. Proces og projektstyring

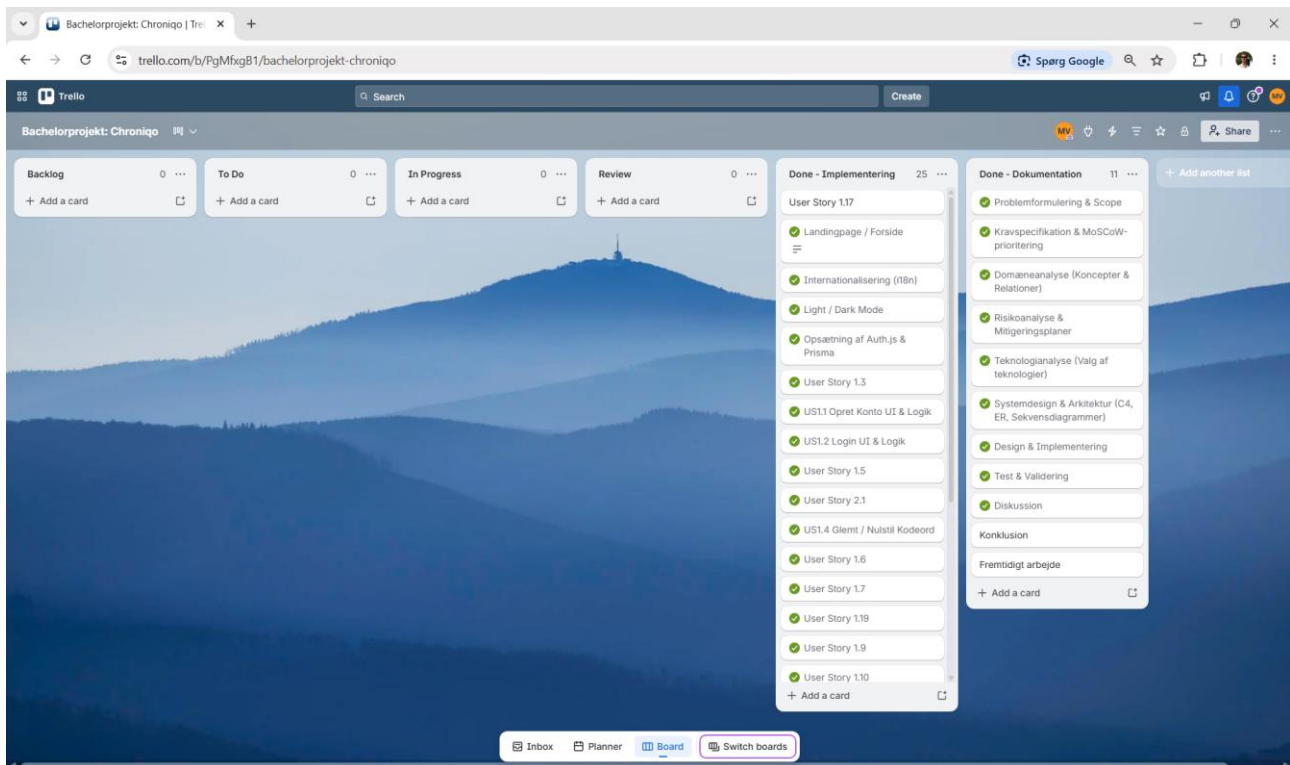
Et projekt efterlader ikke kun kode og dokumentation - det efterlader også et spor af beslutninger, prioriteringer og arbejdsmønstre, der fortæller noget om, hvordan det faktisk blev gennemført. Dette afsnit evaluerer den dimension: om tidsplanen holdt, hvad Trello-boardet og GitLab-aktivitetsgrafen konkret dokumenterer om arbejdsprocessen, og hvad ASE-modellen som udviklingsramme betød i praksis.

8.3.1. Tidsplan og fremdrift

Tidsplanen, der blev præsenteret i afsnit 3.3, holdt i al væsentlighed: rapport, otte bilag og en fuldt deployeret MVP er leveret til tiden. Bufferperioderne var ikke blot en sikkerhedsforanstaltning, men en nødvendighed - i perioder med reduceret kapacitet fungerede MoSCoW-prioriteringen som det daglige kompas for, hvad der absolut skulle løses.

8.3.2. Trello-board som dokumentation for færdiggørelse

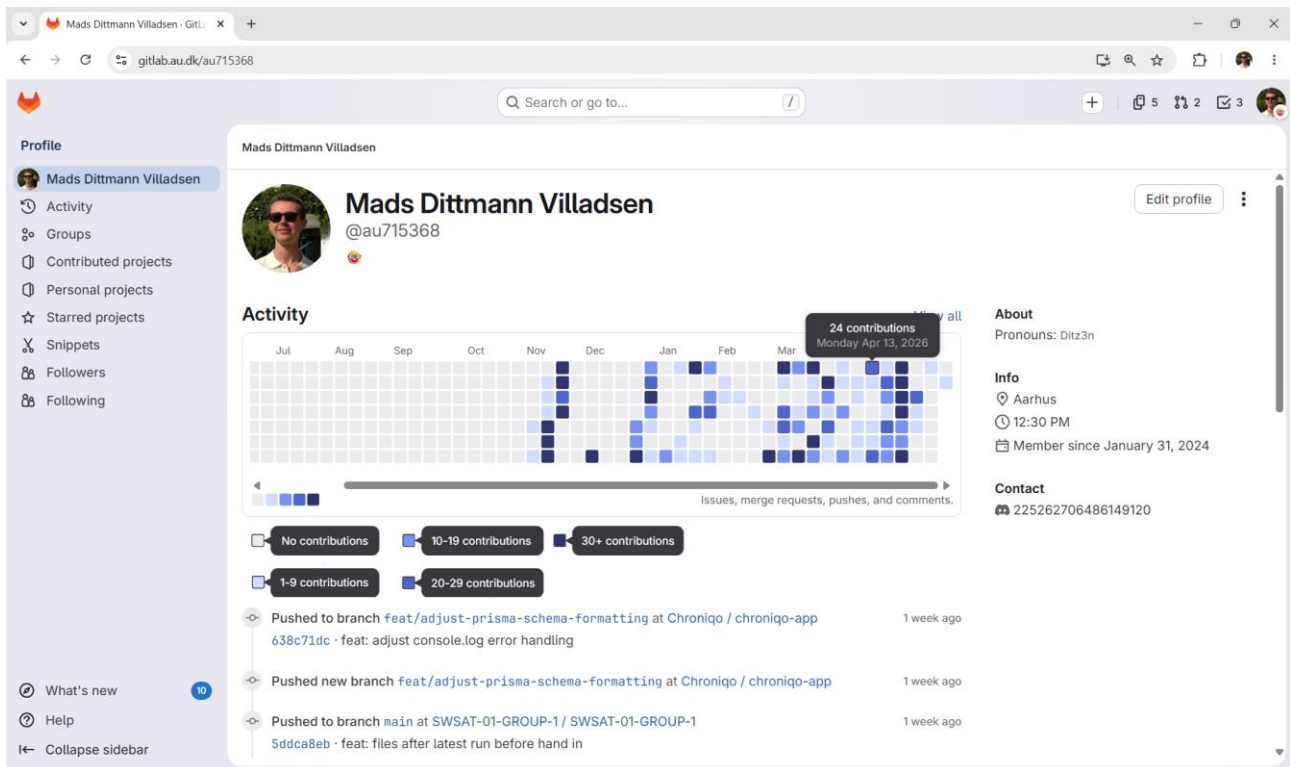
Kanban-tilgangen via Trello-boardet, dokumenteret i Figur 25, viste sig at have en konkret kognitiv funktion, der rakte ud over ren opgavestyring. Det at flytte et kort fra *In Progress* til *Done* på en dag, hvor arbejdskapaciteten var begrænset, fungerede som en håndgribelig bekræftelse af fremdrift - en eksternalisering af overblikket, der reducerede den mentale belastning forbundet med at holde styr på, hvad der manglede.



Figur 25: Skærbillede af Trello-boardet for Chroniqo-projektet ved afleveringstidspunktet. Alle planlagte implementeringsopgaver - User Stories og tekniske milestones - befinder sig i Done - Implementering-kolonnen, mens samtlige dokumentationsopgaver er samlet i Done - Dokumentation-kolonnen. Backlog-, To Do-, In Progress- og Review-kolonnerne er tomme, hvilket dokumenterer, at alle planlagte opgaver er gennemført inden for projektets tidsramme.

8.3.3. GitLab-aktivitetsgraf som dokumentation for udviklingsrytmen

GitLab-aktivitetsgrafen dokumenterer noget, som tal og kravtabeller ikke kan: at udviklingsprocessen ikke har været et sprint op mod deadline, men en vedvarende, disciplineret indsats fordelt over projektets fulde varighed. Commit-historikken, der fremgår af Figur 26, er ikke redigeret til dette formål - den afspejler det faktiske forløb.



Figur 26: GitLab-aktivitetsgraf for Mads Dittmann Villadsens profil, der dækker hele projektperioden fra opstart i slut december 2025 frem til afleveringstidspunktet. Grafen viser en vedvarende og intensiv commit-aktivitet gennem hele perioden, med markante aktivitetsuger fra marts 2026 og frem. Den kontinuerlige aktivitet - herunder på dage med sen- geliggende arbejdstilstand - afspejler en disciplineret og systematisk udviklingsrytme, der har understøttet projektets fremdrift under varierende helbreds-mæssige betingelser.

8.3.4. ASE-modellen i praksis

Valget af ASE-modellen som udviklingsramme (jf. afsnit 3.1 samt Figur 1) viste sig at udgøre en robust metodisk ramme for et enkeltmandsprojekt med denne type kompleksitet. Den iterative kerne - hvor design, implementering og modultest gentages i cyklusser - betød, at testskrivning ikke var en afsluttende aktivitet, men en integreret del af hver iteration. Det fik en konkret konsekvens: regressioner blev fanget tidligt, og den mentale startomkostning ved at genoptage arbejdet efter en pause var markant lavere, fordi testsuiten fungerede som en levende specifikation af systemets forventede adfærd.

8.4. Helbredsmæssige udfordringer og projektets gennemførelse

Chroniqo er bygget til mennesker, der navigerer i hverdagen med et svingende energioverskud. Det er ikke tilfældigt, at det er netop dette projekt, der er valgt - og det er heller ikke tilfældigt, at rammebetingelserne for dets tilblivelse ligner de vilkår, platformen er designet til at håndtere. Ærlighed om den dimension giver den tekniske præstation det korrekte perspektiv.

Projektperioden har været præget af daglige smerter, adskillige hospitalsbesøg og perioder, hvor op til ti timers sengeliggende tilstand om dagen har defineret den reelle arbejdskapacitet. Dertil kommer, at projektet er gennemført sideløbende med en deltidsstilling som student frontend-ingeniør - en balance, der har krævet løbende prioritering og forventningsafstemning med arbejdsgiver på de dage, hvor presset har været for stort til at opretholde begge. I forlængelse af projektets aflevering og mundtligt forsvar er en planlagt operation det næste skridt med henblik på at afhjælpe den fysiske tilstand. Den konkrete og personlige deadline har på sin vis fungeret som en klarhed: der er noget at arbejde hen imod, og det at levere et fuldt gennemarbejdet projekt inden da er en del af den rejse.

Det, der gjorde gennemførelsen mulig, var ikke heroisk viljestyrke, men systematisk ingeniørmæssig tænkning anvendt på selve projektstrukturen. *Architectural Simplification* var ikke udelukkende en teknisk overvejelse - den var en direkte respons på R1-risikoen (jf. afsnit 4.2.1): ved at reducere antallet af bevægelige dele minimerede jeg den kognitive startomkostning ved at genoptage arbejdet. En monolit med én pipeline, én database og ét typelag kræver langt mindre mental optjening end et distribueret system. MoSCoW-prioriteringen fungerede som det daglige kompas: på dage med lavt overskud var det klart, hvad der absolut skulle løses, og hvad der kunne vente til en bedre dag. Kravspecifikationen, teknologianalysen og arkitekturplanen, der var på plads inden første linje produktionskode, betød, at implementeringsarbejdet var et spørgsmål om at bygge det, der allerede var besluttet - ikke om at opfinde arkitekturen undervejs, mens man var træt.

Det eneste element, der ikke kunne realiseres inden for tidsrammen, er brugertesten. Det er den rigtige afvejning: brugertesten er den mest ressourcekrævende aktivitet at koordinere logistisk - rekruttering fra patientforeninger, aftaleindgåelse, facilitering - og dens fravær svækker valideringsgrundlaget, men ikke systemets tekniske korrekthed. Protokollen er fuldt specificeret og klar til eksekvering.

Den overordnede læringspointe er imidlertid af en anden karakter. Det, jeg tager med mig fra dette projekt, er værdien af grundig analyse og dokumentation som forudsætning for effektiv implementering. Kravspecifikationen, domæneanalysen, risikoanalysen, teknologianalysen og systemarkitekturen var ikke formalia - de var en byggeplan. At have planen på plads, inden der skrives en linje produktionskode, er forskellen på at bygge og på at finde ud af, hvad man bygger, mens man er ved det. Den skelnen er langt mere afgørende i et projekt med begrænset kapacitet end i et projekt med ubegrænsede ressourcer.

9. Konklusion

Dette projekt tog sit udgangspunkt i en dokumenteret mangel: eksisterende sociale platforme er designet ud fra en præmis om konstant tilgængelighed og aktivt engagement, der systematisk ekskluderer brugere, hvis kroniske sygdom begrænser deres daglige energioverskud. Projektets centrale problemformulering spurgte, hvordan softwareteknologi kan anvendes til at mindske social isolation hos denne gruppe ved at udvikle en webapplikation, der adapterer dele af brugeroplevelsen efter individets aktuelle dagsform. Svaret har været at lade dagsformen udgøre systemets centrale tilstandsvariabel - ikke som et statistisk datapunkt, men som den mekanisme, der driver interfacets adfærd i realtid.

Det første underspørgsmål spurgte, hvordan dagsformsindikatorer kan implementeres som en central logik, der automatisk adapterer brugergrænsefladen uden at øge den kognitive kompleksitet for slutbrugeren. Løsningen er centraliseret i `DailyStatusContext`, der distribuerer det registrerede energiniveau til hele applikationstræet via React Context API. Når brugeren registrerer et lavt energiniveau (niveau 0 eller 1, nulindekseret), aktiverer systemet automatisk *Low Energy Mode*: notifikationsbadges skjules for at fjerne opfordringen til øjeblikkelig handling, og uendeligt scroll erstattes af paginering for at modvirke passivt doomscrolling (jf. afsnit 6.1.4). Det adaptive skift er fuldautomatisk og forudsætter ingen aktiv handling fra brugeren ud over selve dagsformsregistreringen - den kognitive kompleksitet øges dermed ikke, men reduceres. User Stories US2.1 og US2.13 er begge godkendt ved manuel accepttest (jf. afsnit 7.4).

Det andet underspørgsmål spurgte, hvilke tekniske valg i frontend-udviklingen der bedst understøtter en responsiv brugerflade til målgruppen. Det afgørende valg var *Architectural Simplification*: Next.js-monolittens samlede arkitektur (jf. afsnit 4.3.2 og afsnit 5) eliminerede CORS-kompleksitet, muliggjorde end-to-end-typesikkerhed via Prisma og reducerede antallet af komponenter i infrastrukturen til et minimum. Kombinationen af Tailwind CSS v4's design tokens og platformens komponentbibliotek gjorde det muligt at implementere det adaptive interface med et konsistent og farvekodet visuelt system for dagsform, der er forankret i `globals.css` og ikke kræver specialiseret viden at vedligeholde (jf. afsnit 6.1 og 6.3). Det var netop denne arkitektoniske forenkling, der frigjorde kapacitet til at bygge de adaptive UX-mekanismer, der udgør platformens primære differentiering.

Det tredje underspørgsmål spurgte, hvordan man kan validere, at løsningen reelt imødekommer kravene til et lavenergi-fællesskab. Verificering er gennemført på tre tekniske niveauer: 858 automatiserede Jest-tests gennemføres succesfuldt i GitLab CI/CD-pipelinen og verificerer robustheden i systemets forretningslogik; databasetesten via Prisma Studio bekræfter ACID-compliance og korrekt kaskadeadfærd; og den manuelle accepttest godkender samtlige 68 User Stories på tværs af de fire Epics (jf. afsnit 7). Den empiriske brugervalidering - det fjerde niveau i V-modellen (jf. Figur 23) - er fuldt protokolleret med Think-Aloud-metode, målrettet deltagerprofil og kvantificerbare succes-kriterier, men var ikke mulig at eksekvere inden for projektets tidsramme som følge af de helbreds-mæssige udfordringer beskrevet i afsnit 8.4.

På baggrund af de tre besvarede underspørgsmål kan projektets centrale problemformulering besvares bekræftende: softwareteknologi kan aktivt anvendes til at mindske social isolation hos kronisk syge ved at lade dagsformen drive applikationens adaptive adfærd. Chroniqo leverer som MVP et fuldt implementeret og verificeret system, hvor registrering af energiniveau, automatisk UI-adaption, anonymt fællesskab, asynkron interaktion og AI-støttebotten Niqo udgør en sammenhængende platform, der er designet til at tilpasse sig brugerens aktuelle kontekst og behov frem for at tvinge brugeren ind i faste systemrammer. Den ene afgørende åbne hypotese er, hvorvidt *Low Energy Mode* opleves som kognitivt skånende i praksis af brugere fra målgruppen. Mens den tekniske verifikation bekræfter, at mekanismen fungerer korrekt, kan kun de forestående brugertests validere den oplevede værdi. Dette udgør det centrale udgangspunkt for det fremtidige arbejde, som udfoldes i afsnit 10.

10. Fremtidigt arbejde

Chroniqo er leveret som en fuldt funktionel MVP, der opfylder samtlige definerede Must-have-krav. MVP-tilgangen er et bevidst udviklingsparadigme: et afgrænset, stabilt system leveres først, og videreudviklingen prioriteres iterativt på baggrund af reelle brugererfaringer. Dette afsnit identificerer de indsatsområder, der udgør næste iteration - enten fordi de er dokumenterede tekniske kompromiser, eksplicitte afgrænsningsvalg fra afsnit 1.3 eller åbne valideringspunkter fra afsnit 7.

10.1. Eksekvering af brugertest med målgruppen

Den mest kritiske opgave er eksekveringen af den brugertest, der af helbredsmæssige årsager ikke kunne gennemføres inden for projektets tidsramme (jf. afsnit 7.6).

Testprotokollen er fuldt specificeret og klar til umiddelbar eksekvering som en kvalitativ Think-Aloud-test med fem deltagere rekrutteret fra relevante patientforeninger. Det centrale succeskrITERIUM - at minimum 4 ud af 5 deltagere spontant bemærker den visuelle ændring ved *Low Energy Mode* uden vejledning (US2.13, UR4) - repræsenterer den kernehypotese, hele systemets adaptive design hviler på.

Kun brugervalideringen kan bekræfte, at løsningen opleves som meningsfuld af målgruppen, og resultaterne vil danne det empiriske grundlag for efterfølgende UI/UX-iterationer.

10.2. Opgradering til Managed Realtime-chatinfrastruktur

Chatfunktionaliteten er implementeret via short-polling som et bevidst arkitektonisk kompromis, begrundet i serverless-infrastrukturens manglende understøttelse af persistente TCP-forbindelser (jf. afsnit 8.2.2). Kompromisset er acceptabelt i MVP-fasen, men udgør en skaleringsbarriere ved stigende antal samtidige brugere.

En fremtidig iteration bør erstatte short-polling med en Managed Realtime Service - Pusher eller Supabase Realtime er begge vurderede kandidater i Bilag 4 (20) - der håndterer realtidstrafik eksternt og aflaster PostgreSQL markant. Migreringen kan gennemføres inkrementelt via den modulare Next.js App Router-struktur og vil samtidig skabe grundlag for formel load-test af PR2.

10.3. Native mobilapplikation

Chroniqo er bevidst begrænset til en responsiv webapplikation (afsnit 1.3.1). For en målgruppe, der typisk tilgår sociale platforme fra mobile enheder med begrænset energi, er forskellen på et responsiv weblayout og en dedikeret native oplevelse mærkbar.

En React Native-applikation via Expo-frameworket er det oplagte næste skridt; Expo giver adgang til push-notifikations-API'et, der vil muliggøre dagsformstilpassede notifikationer. Den bagvedliggende infrastruktur forbliver i vidt omfang intakt, da en native applikation kommunikerer med de eksisterende API-endpoints.

10.4. WCAG 2.1 AA-certificering

FURPS+-krav UR3 er vurderet som delvist opfyldt (jf. afsnit 7.5): WCAG 2.1 AA-overensstemmelse er bekræftet for den primære navigationsflow, men en systematisk audit af samtlige komponenter er ikke gennemført.

En formel audit - f.eks. via axe DevTools (64) eller ekstern tilgængelighedsspecialist - ville kortlægge resterende mangler og give et troværdigt certificeringsgrundlag.

For en platform rettet mod kronisk syge brugere, der hyppigt oplever komorbiditet²⁷ med visuelle eller motoriske vanskeligheder, er certificeringen ikke blot juridisk compliance (50), men et direkte udtryk for platformens kerneværdi.

²⁷ Comorbidity.

10.5. Niqo med dagsforms-integration og opt-in langtidshukommelse

Den nuværende implementering af Niqo er sessionsbaseret og stateless i den forstand, at dagsforms-data ikke indgår i Gemini API-kaldet.

En væsentlig udvidelse ville være en opt-in mekanisme, der tillader Niqo at inkorporere brugerens dagsformshistorik i sin kontekst: en bruger, der konsekvent har registreret dagsform 1 (Helt Udmattet) de seneste syv dage, giver Niqo et kvalitativt anderledes afsæt for empatisk støtte. Dagsforms-data er allerede tilgængeligt server-side og kan inkluderes i Gemini-kaldet inden afsendelse, mens Privacy by Design fastholdes via opt-in og det eksisterende PII-filter.

En sådan udvidelse vil transformere Niqo fra generisk AI-chatinterface til et personaliseret støtteværktøj, der reflekterer brugerens faktiske energimønstre over tid.

10.6. Formel load-test og optimering af server-side rendering i feed

To performancerelaterede indsatsområder er identificeret i afsnit 7.5:

PR2 (50 samtidige brugere) er ikke formelt load-testet inden for MVP-scopet; arkitektonisk er kravet adresseret via Vercels horisontale skalering og Upstash Redis, men formel verifikation kræver et dedikeret testmiljø som k6 eller Artillery (65, 66).

Feed-indlæsningen (PR1) er delvist opfyldt: den nuværende client-side SWR-vandfaldsstruktur kan adresseres via Server-Side Rendering af den initielle feed-payload, så det første skærmindhold leveres i den initielle HTML-respons - en omstrukturering til Incremental Static Regeneration (ISR) (67), der pre-renderer feed-indholdet ved build- eller revalideringstidspunktet, eller Server Components med React Suspense (68) er den teknisk veldefinerede vej frem.

11. Referencer

1. Fowler M. Ubiquitous Language [Internet]. 31. oktober 2006. [hentet 27. januar 2026]. Tilgængelig fra: <https://martinfowler.com/bliki/UbiquitousLanguage.html>.
2. Bannon S, Greenberg J, Mace RA, Locascio JJ, Vranceanu AM. The role of social isolation in physical and emotional outcomes among patients with chronic pain. *Gen Hosp Psychiatry*. 2021;69:50–4. [hentet 28. januar 2026]. Tilgængelig fra: <https://pmc.ncbi.nlm.nih.gov/articles/PMC7979493/>.
3. Iovino P, Vellone E, Cedrone N, Riegel B. A Middle-Range Theory of Social Isolation in Chronic Illness. *Int J Environ Res Public Health*. 2023;20(6). [hentet 28. januar 2026]. Tilgængelig fra: <https://pmc.ncbi.nlm.nih.gov/articles/PMC10049704/>.
4. Sundhedsstyrelsen. Kroniske smerter [Internet]. Sundhedsstyrelsen; [år ukendt]. [hentet 28. januar 2026]. Tilgængelig fra: <https://www.sst.dk/kroniske-smerter>.
5. Harris T. How Technology Hijacks People's Minds — from a Magician and Google's Design Ethicist [Internet]. 18. maj 2016. [hentet 28. januar 2026]. Tilgængelig fra: <https://medium.com/thrive-global/how-technology-hijacks-peoples-minds-from-a-magician-and-google-s-design-ethicist-56d62ef5edf3>.
6. Thompson DM, Booth L, Moore D, Mathers J. Peer support for people with chronic conditions: a systematic review of reviews. *BMC Health Serv Res*. 2022;22(1):427. [hentet 28. januar 2026]. Tilgængelig fra: <https://pubmed.ncbi.nlm.nih.gov/35361215/>.
7. Aarhus University. Development Processes. Forelæsningsslides. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2023.
9. Agile Business Consortium. Chapter 10: MoSCoW Prioritisation - DSDM Project Framework Handbook [Internet]. [år ukendt]. [hentet 5. februar 2026]. Tilgængelig fra: <https://www.agilebusiness.org/dsdm-project-framework/moscow-prioritisation.html>.
10. Ziemek M. Documenting non-functional requirements using FURPS+ [Internet]. 2022. [hentet 5. februar 2026]. Tilgængelig fra: https://marcinziemek.com/blog/content/articles/8/article_en.html.
11. Martin RC. Agile Software Development: Principles, Patterns, and Practices [Internet]. 2003. [hentet 6. februar 2026]. Tilgængelig fra: <https://dl.acm.org/doi/10.5555/515230>.
12. Datatilsynet. Databeskyttelse gennem design og standardindstillinger [Internet]. [år ukendt]. [hentet 3. marts 2026]. Tilgængelig fra: <https://www.datatilsynet.dk/regler-og-vejledning/behandlingssikkerhed/databeskyttelse-gennem-design-og-standardindstillinger>.
13. Schwaber K, Sutherland J. The Scrum Guide - The Definitive Guide to Scrum: The Rules of the Game [Internet]. 2020. [hentet 6. februar 2026]. Tilgængelig fra: <https://scrumguides.org/scrum-guide.html>.

14. Anderson DJ, Carmichael A. The Official Guide to The Kanban Method [Internet]. 2021. [hentet 6. februar 2026]. Tilgængelig fra: <https://kanban.university/kanban-guide/>.
15. Hanssen GK, Bjørnson FO, Westerheim H. Tailoring and Introduction of the Rational Unified Process [Internet]. 2007. [hentet 6. februar 2026]. Tilgængelig fra: https://www.academia.edu/14733833/Tailoring_and_introduction_of_the_rational_unified_process.
16. GitLab Inc. GitLab Documentation [Internet]. [år ukendt]. [hentet 10. februar 2026]. Tilgængelig fra: <https://docs.gitlab.com/ci/>.
19. The OWASP Foundation. HttpOnly [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://owasp.org/www-community/HttpOnly>.
21. Microsoft Corporation. TypeScript Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://www.typescriptlang.org/docs/>.
22. Vercel. Vercel Documentation [Internet]. [år ukendt]. [hentet 23. februar 2026]. Tilgængelig fra: <https://vercel.com/docs>.
23. Vercel. Next.js Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://nextjs.org/docs>.
24. Tailwind Labs. Getting started with Tailwind CSS [Internet]. [år ukendt]. [hentet 24. februar 2026]. Tilgængelig fra: <https://tailwindcss.com/docs>.
25. Prisma Data. Prisma Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://www.prisma.io/docs>.
26. PostgreSQL Global Development Group. PostgreSQL 18.2 Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://www.postgresql.org/docs/18.2/index.html>.
27. Neon. Neon Documentation [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://neon.com/docs/introduction>.
28. Redis Inc. Redis Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://redis.io/docs/latest/>.
29. Upstash Inc. Upstash - Serverless Data Platform [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://upstash.com/>.
30. Auth.js contributors. Authjs: Documentation [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://authjs.dev/getting-started>.
31. Parecki A. Protecting Apps with PKCE [Internet]. [år ukendt]. [hentet 22. februar 2026]. Tilgængelig fra: <https://www.oauth.com/oauth2-servers/pkce/>.
32. Jones MB, Bradley J, Sakimura N. JSON Web Token (JWT) (RFC 7519) [Internet]. 2015. [hentet 25. februar 2026]. Tilgængelig fra: <https://datatracker.ietf.org/doc/html/rfc7519>.

33. Svix. Long Polling vs Short Polling [Internet]. [år ukendt]. [hentet 23. februar 2026]. Tilgængelig fra: <https://www.svix.com/resources/faq/long-polling-vs-short-polling/>.
34. Google. Google AI Studio and Gemini API Documentation [Internet]. [år ukendt]. [hentet 23. februar 2026]. Tilgængelig fra: <https://ai.google.dev/gemini-api/docs>.
35. Docker. Docker Documentation [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://docs.docker.com/>.
36. OpenJS Foundation. Jest Documentation [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://jestjs.io/docs/getting-started>.
37. marchaos. Type safe mocking extensions for Jest [Internet]. [år ukendt]. [hentet 25. februar 2026]. Tilgængelig fra: <https://www.npmjs.com/package/jest-mock-extended>.
39. Brown S. The C4 model for visualising software architecture [Internet]. [år ukendt]. [hentet 27. februar 2026]. Tilgængelig fra: <https://c4model.com/>.
40. Vercel. Next.js: Edge Runtime [Internet]. [år ukendt]. [hentet 1. marts 2026]. Tilgængelig fra: <https://nextjs.org/docs/app/api-reference/edge>.
41. The OWASP Foundation. Insecure Direct Object Reference Prevention Cheat Sheet - OWASP Cheat Sheet Series [Internet]. [år ukendt]. [hentet 3. marts 2026]. Tilgængelig fra: https://cheatsheetseries.owasp.org/cheatsheets/Insecure_Direct_Object_Reference_Prevention_Cheat_Sheet.html.
42. Europa-Parlamentet og Rådet for Den Europæiske Union. Europa-Parlamentets og Rådets forordning (EU) 2016/679 af 27. april 2016 om beskyttelse af fysiske personer i forbindelse med behandling af personoplysninger og om fri udveksling af sådanne oplysninger (databeskyttelsesforordningen) [Internet]. 2016. [hentet 3. marts 2026]. Tilgængelig fra: <https://eur-lex.europa.eu/legal-content/DA/TXT/?uri=CELEX%3A32016R0679>.
43. Richards M. Software Architecture Patterns - Layered Architecture [Internet]. 2015. [hentet 2. marts 2026]. Tilgængelig fra: <https://www.oreilly.com/content/software-architecture-patterns/>.
44. Fowler M. Data Transfer Object [Internet]. 2004. [hentet 8. marts 2026]. Tilgængelig fra: <https://martinfowler.com/eaCatalog/dataTransferObject.html>.
45. McDonnell C. Zod: TypeScript-first schema validation with static type inference [Internet]. [år ukendt]. [hentet 8. marts 2026]. Tilgængelig fra: <https://zod.dev/>.
46. Fielding RT. Architectural Styles and the Design of Network-based Software Architectures [Internet]. 2000. [hentet 9. marts 2026]. Tilgængelig fra: https://roy.gbiv.com/pubs/dissertation/rest_arch_style.html.
47. MDN Web Docs. HTTP response status codes [Internet]. [år ukendt]. [hentet 9. marts 2026]. Tilgængelig fra: <https://developer.mozilla.org/en-US/docs/Web/HTTP/Status>.

50. W3C. Web Content Accessibility Guidelines (WCAG) 2.1. W3C Recommendation [Internet]. 6. maj 2025. [hentet 10. april 2026]. Tilgængelig fra: <https://www.w3.org/TR/WCAG21/>.
51. Google. Google Fonts: Poppins [Internet]. [år ukendt]. [hentet 15. marts 2026]. Tilgængelig fra: <https://fonts.google.com/specimen/Poppins?specimen.preview.text=Chroniqo>.
52. Google. Google Fonts: Inter [Internet]. [år ukendt]. [hentet 15. marts 2026]. Tilgængelig fra: <https://fonts.google.com/specimen/Inter>.
53. Dansk Sprognævn. Nye ord i dansk - doomscrolling [Internet]. 2020. [hentet 20. marts 2026]. Tilgængelig fra: <https://noid.dsn.dk/ordbog/doomscrolling/>.
54. Meta Platforms I. React - createContext [Internet]. [år ukendt]. [hentet 22. marts 2026]. Tilgængelig fra: <https://react.dev/reference/react/createContext>.
56. Fowler M. Fail Fast [Internet]. 2004. [hentet 25. marts 2026]. Tilgængelig fra: <https://martinfowler.com/ieeeSoftware/failFast.pdf>.
57. Upstash Inc. Upstash Ratelimit: Rate Limiting Algorithms [Internet]. [år ukendt]. [hentet 1. marts 2026]. Tilgængelig fra: <https://upstash.com/docs/oss/sdks/ts/ratelimit/algorithms>.
58. The OWASP Foundation. JSON Web Token for Java Cheat Sheet - OWASP Cheat Sheet Series [Internet]. [år ukendt]. [hentet 10. marts 2026]. Tilgængelig fra: https://cheatsheetseries.owasp.org/cheatsheets/JSON_Web_Token_for_Java_Cheat_Sheet.html.
60. Aarhus University. Systemtest - Introduction to Systems Engineering I2ISE. Forelæsningslides. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2023.
61. Nielsen J. Thinking Aloud: The #1 Usability Tool [Internet]. Nielsen Norman Group; 2012. [hentet 2. april 2026]. Tilgængelig fra: <https://www.nngroup.com/articles/thinking-aloud-the-1-usability-tool/>.
62. Microsoft Corporation. Unit testing best practices with .NET - Arrange, Act, Assert [Internet]. [år ukendt]. [hentet 2. april 2026]. Tilgængelig fra: <https://learn.microsoft.com/en-us/dotnet/core/testing/unit-testing-best-practices>.
63. Nielsen J. Why You Only Need to Test with 5 Users [Internet]. Nielsen Norman Group; 18. marts 2000. [hentet 2. april 2026]. Tilgængelig fra: <https://www.nngroup.com/articles/why-you-only-need-to-test-with-5-users/>.
64. Deque Systems. axe DevTools - Accessibility Testing [Internet]. [år ukendt]. [hentet 10. april 2026]. Tilgængelig fra: <https://www.deque.com/axe/devtools/>.
65. Grafana Labs. k6: Modern load testing for developers [Internet]. [år ukendt]. [hentet 28. april 2026]. Tilgængelig fra: <https://k6.io/>.
66. Artillery.io Ltd. Artillery: Cloud-scale performance testing [Internet]. [år ukendt]. [hentet 28. april 2026]. Tilgængelig fra: <https://www.artillery.io/>.

67. Vercel. Next.js: How to implement Incremental Static Regeneration (ISR) [Internet]. [år ukendt]. [hentet 15. maj 2026]. Tilgængelig fra: <https://nextjs.org/docs/pages/guides/incremental-static-regeneration>.
68. Meta Platforms I. React - Suspense [Internet]. [år ukendt]. [hentet 15. maj 2026]. Tilgængelig fra: <https://react.dev/reference/react/Suspense>.

12. Bilag

8. Villadsen MD. Chroniqo: Kravspecifikation. Bilag 1. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.
17. Villadsen MD. Chroniqo: Domæneanalyse. Bilag 2. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.
18. Villadsen MD. Chroniqo: Risikoanalyse. Bilag 3. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.
20. Villadsen MD. Chroniqo: Teknologianalyse. Bilag 4. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.
38. Villadsen MD. Chroniqo: Systemdesign & Arkitektur. Bilag 5. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.
48. Villadsen MD. Chroniqo: Design & Implementering. Bilag 6. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.
49. Villadsen MD. Chroniqo: Kildekode. Bilag 9. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.
55. Villadsen MD. Chroniqo: UI-Dokumentation. Bilag 8. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.
59. Villadsen MD. Chroniqo: Test & Validering. Bilag 7. Aarhus Universitet, Institut for Elektro- og Computerteknologi; 2026.